

A Hierarchical ReAct Multi-Agent Pipeline for Residential Appliance Load Scheduling Under Dynamic Electricity Pricing

University of Oulu
Information Processing Science
Master's Thesis
Hoan Le
2026

Abstract

Households on dynamic electricity contracts can lower their bills by running flexible appliances in low-price hours, but coordinating several appliances under a volatile price curve and a household peak load limit is difficult without automation. This thesis follows Design Science Research to design and evaluate a hierarchical ReAct multi-agent system, built in LangGraph, that gives a household a daily appliance recommendation. Two language model agents share the work: an Orchestrator reads the day's appliance plan and explains the final schedule, and a Price-Aware Scheduler plans the start times with tools, among them an optimiser that solves the whole day. The steps that need no language model are done by a deterministic harness: it builds the appliance profiles from the household data, retrieves similar solved days as worked examples, and checks every schedule against the windows and the peak limit, returning any problem to the Scheduler once. To test whether the hierarchy matters, the same harness also runs a single agent with the same tools. The study asks how much the recommendations save, how often they are valid and what they cost to produce compared with a language model answering without tools (SQ1), and under which price patterns and household loads they save at least as much as the home energy management tools households use today: cheapest-hours automations and the open-source optimiser EMHASS (SQ2).

The evaluation combines appliance cycles and usual run times from the REFIT dataset with Finnish Nord Pool prices. Every scenario keeps the household load within 8 kW, the threshold of the Finnish power charge. A factorial benchmark varies four price patterns and three household loads, from two appliances to a household that also charges an electric car and heats a sauna. The design was developed on exploratory scenarios, and nine pre-registered hypotheses are tested on 72 held-out scenarios with fifteen-minute prices. The primary outcome is the saving against starting every appliance when it is ready. On the held-out scenarios, the pipeline with GPT-6 Luna produced a valid schedule in all 216 runs and saved 15.9 % of the cost, the same as the optimal schedule, against 4.1 % for the same model without tools, which was valid in 77 % of its runs. A recommendation took 7.4 seconds on average and cost $0.0004 in API fees. The pipeline saved 4.6 and 6.0 percentage points more per scenario than the cheapest-hours automations, which plan each appliance on its own and broke the 8 kW limit on up to 46 % of the days with a car and a sauna, and the same as EMHASS. Seven of the nine hypotheses were supported; the advantage over the automations was not significantly larger with the electrified load. A single agent in the same harness returned the same schedules a third faster and at 60 % of the API cost, and a simple peak-aware rule written for this study also reached the optimum. Reliability came from the solver and the deterministic checks rather than from the number of agents.

Keywords

multi-agent system, ReAct, LangGraph, retrieval-augmented generation, home energy management, appliance scheduling, dynamic electricity pricing, Design Science Research

Supervisor

Yueqiang Xu, PhD, Postdoctoral Researcher

Foreword

This thesis was written for the Master's programme in Information Processing Science at the University of Oulu. The idea for it came from my traineeship at the university, where I worked on an AI-assisted service that tells Finnish households when to shift their electricity use. That work left me with a question I wanted to study properly: can language model agents give a household advice on when to run its appliances that is worth following, and what does that advice cost to produce?

I thank my supervisor, Yueqiang Xu, for his guidance and feedback. The study relies on open data and software: the REFIT appliance measurements of the University of Strathclyde, the electricity prices published on the ENTSO-E Transparency Platform and by Elering, and the open-source EMHASS optimizer.

Oulu, October 2026

Hoan Le

Abbreviations

API	Application Programming Interface
DSR	Design Science Research
HEMS	Home Energy Management System
LLM	Large Language Model
MAS	Multi-Agent System
MCP	Model Context Protocol
MILP	Mixed Integer Linear Programming
Nord Pool FI	Nord Pool Finland Bidding Zone
PRISMA	Preferred Reporting Items for Systematic reviews and Meta-Analyses
RAG	Retrieval-Augmented Generation
ReAct	Reasoning and Acting
REFIT	Personalised Retrofit Decision Support Tools for UK Homes using Smart Home Technology (the research project that produced the dataset)
SLM	Small Language Model

Declaration of Generative AI in the Thesis

I used the generative AI tool Claude (Anthropic) in this thesis work: to help write and test the software of the evaluated system, to help search and screen the literature reported in Appendix A, and to help draft and revise the text of the thesis. The language models GPT-6 Luna and gpt-oss-20b are objects of study in this thesis and were not used to write it. All content produced with the tool was reviewed and edited by me, and I take full responsibility for the accuracy and integrity of the final work.

Contents

Abstract 1

Foreword 3

Abbreviations 4

Declaration of Generative AI in the Thesis 5

1 Introduction 9

1.1 Background and motivation 9

1.2 Research problem 10

1.3 Design Science Research approach 11

1.4 Structure of the thesis 11

2 Literature Review 12

2.1 Home energy management systems 12

2.2 LLM-based agents for energy management 13

2.3 The ReAct framework 16

2.4 Multi-agent LLM systems 16

2.5 Retrieval and solver tools for LLM agents 17

2.6 Finnish energy market context 18

2.7 Literature synthesis and research gap 18

3 Research Methodology 21

3.1 Design Science Research methodology 21

3.2 Theoretical foundations 22

3.3 Evaluation strategy 23

3.3.1 Data collection 26
3.3.2 Data analysis 27

4 System Design and Implementation 30

4.1 Problem definition and objectives 30
4.2 Architecture overview 30
4.3 Agent specifications 32
4.4 Retrieval of solved days and the optimiser tool 35
4.5 MILP solver 36
4.6 Technology stack 37

5 Evaluation and Results 39

5.1 Dataset and scenario design 39
5.1.1 Appliance data 39
5.1.2 Electricity price data 40
5.1.3 Scenario construction 41
5.1.4 Factorial benchmarks 43
5.2 Value of 15-minute price information 43
5.3 Demonstration results 44
5.3.1 Design iterations 46
5.4 Exploratory results 48
5.5 Confirmatory evaluation on the held-out set Q3 49

6 Discussion 52

6.1 Interpretation of the research questions 52
6.1.1 RQ: design of the multi-agent system 52
6.1.2 SQ1: value and cost of the recommendations 53
6.1.3 SQ2: conditions under which the system pays off 53
6.2 Relation to prior work 54

6.3 Implications for practice 55
6.4 Limitations 55
6.5 Future research 57

7 Conclusions **58**
7.1 Answers to the research questions 58
7.2 Contributions 59
7.3 Implications 59
7.4 Limitations and further work 59

References **61**

A Literature search **69**

B Eligible studies **71**

1. Introduction

Smart meters, dynamic electricity contracts and a growing share of renewable generation have changed what a household pays for electricity from hour to hour. A household on a spot-price contract can lower its bill by running flexible appliances, such as the washing machine or the dishwasher, in the day's low-price hours. Without automation this is hard when several appliance cycles have to fit a price curve that changes every day and the household must also stay under a limit on its total power.

Finland is a clear case. Since October 2025, Finnish consumers on spot-price contracts pay prices set for every 15 minutes on the Nordic spot market (Nord Pool FI bidding zone), and prices within a day vary widely. From 2023 to 2026, the most expensive hour of a day cost a median of 3.1 times the cheapest on days without near-zero prices, and one day in five had at least one hour with a negative price (computed from Nord Pool FI prices; Pakastin Oy, 2026). Such extreme prices measurably change household consumption and affect the distribution network (Haakana et al., 2025). Language models have recently been applied to the same problem: in the first Home Energy Management Systems (HEMS) built on large language models, agents schedule household appliances, a task that earlier needed hand-written rules or an optimization solver (El Makroum, Zwickl-Bernhard, & Kranzl, 2026; Jonah et al., 2026). These first systems have documented limitations, which this thesis takes up.

1.1 Background and motivation

HEMS have mostly been built on mathematical optimization, rule-based schedulers, or machine learning models trained on past consumption (Beaudin & Zareipour, 2015). These approaches work well for a precisely specified problem, but they take expert effort to configure, and a rule or a trained model does not carry over to price conditions it was not designed for.

Large Language Models (LLMs) can be used as general reasoning engines, which gives HEMS design another option. Yao et al. (2023) introduce ReAct, a Thought-Action-Observation loop in which the model alternates between reasoning and tool calls, and show that it solves multi-step tasks that need information looked up along the way. For household scheduling, a ReAct agent can read the price forecast, decide which appliances can be shifted, find the cheapest time window for each and check the result against the household's constraints, and every step of that reasoning is recorded.

A secondary motivation concerns sustainable AI deployment. The energy needed per inference differs by orders of magnitude between models and tasks (Luccioni et al., 2024), and serving a small language model (SLM) of about 7 billion parameters has been estimated to be 10 to 30 times cheaper in latency, energy and computation than serving a

model of 70 to 175 billion parameters (Belcak et al., 2025). Whether such smaller models are good enough for a given agentic task is an empirical question. Each recommendation requires several calls to a language model, which costs money and energy. The inference cost of each design, measured in tokens and response time, is therefore reported alongside the saving it achieves. A design that reaches the same saving with fewer model calls is cheaper to run and easier to move to a small local model later.

1.2 Research problem

In the hierarchical multi-agent system of El Makroum, Zwickl-Bernhard, and Kranzl (2026), only one of three evaluated models coordinated all appliances, and the evaluation covers a single day. Reviewing that work, Jonah et al. (2026) list multi-appliance coordination failures, scheduling on price alone, a single-day evaluation horizon and untested constraint safety as open problems. Their own single-agent system is evaluated for one household over a few weeks, and they recommend that every schedule be validated independently before it is executed. Neither study varies the price conditions. It is also unclear what a language model agent adds over the tools households can already use, such as cheapest-hours automations and open-source optimizers, and whether an agent can match an optimizer when several appliances have to be staggered. Two techniques used in other LLM systems could help but have not been tried for household scheduling: giving the model retrieved examples of solved problems (retrieval-augmented generation, RAG), and giving it a solver as a tool.

The artifact studied in this thesis gives each household a daily recommendation. Every day it receives the household's appliance plan for that day, namely which appliance cycles are to run and when each is ready and must be finished, together with the household's base load, its peak load limit and the day's electricity prices. It returns a start time for each appliance and a short explanation. To address the limitations above, the study is guided by one main research question and two sub-questions:

> **RQ:** How can a hierarchical ReAct multi-agent system (the Multi-Agent Pipeline) be designed to give households reliable, cost-saving daily appliance recommendations under dynamic electricity prices?
>
> **SQ1 (value and cost):** How much do the pipeline's recommendations save, how often are they valid, and what do they cost to produce, compared with a language model that answers without tools?
>
> **SQ2 (conditions):** Under which price patterns and household loads do the pipeline's recommendations save at least as much as existing home energy management tools?

The main RQ states the design aim: recommendations that are reliable, meaning that they respect every appliance window and the household peak limit, and that save money against starting every appliance when it is ready. The pipeline studied is the final design:

two ReAct agents, an Orchestrator and a Price-Aware Scheduler, inside a deterministic harness that profiles the appliances, shows the Scheduler how similar past days were solved best and checks every schedule. The Scheduler hands the search to an optimiser tool, and the Orchestrator explains the result. Part of the RQ is whether the hierarchy itself matters, so the pipeline is also compared with a single agent in the same harness. SQ1 concerns what its recommendations are worth and what they cost. It tests the premise of the ReAct approach, that reasoning grounded in tool observations does better than unaided generation, by comparing the pipeline with the same language model answering directly, and it checks the response time and API cost of each recommendation against the design objectives. SQ2 concerns where the system is worth using. Its ”existing home energy management tools” are real, freely available software that a household can install without programming: the cheapest-hours automations shared for Home Assistant, which switch a device on in the day’s cheapest hours, and the open-source optimizer EMHASS, which computes a schedule for the whole household. SQ2 compares the recommendations with these tools and asks how the comparison depends on the day’s prices and on the household’s load. The peak limit is the 8 kW threshold above which Finnish network companies may charge for power (Section 5.1). The REFIT laundry and dishwasher loads rarely reach it, so the heaviest load adds an electric car charge and a sauna, the loads that make Finnish houses compete for capacity.

1.3 Design Science Research approach

This study follows the Design Science Research (DSR) methodology as defined by Peffers et al. (2007). The study builds a software artifact, the hierarchical ReAct multi-agent system, and evaluates it over several iterations, which is the kind of work DSR is meant for. As Hevner et al. (2004) explain, DSR produces knowledge about a design problem and its solution by building and evaluating an artifact. The artifact is the main research contribution. A structured literature search, reported with a PRISMA 2020 flow diagram (Appendix A), forms DSR Phase 1: it identifies the research gap and grounds the architectural decisions in prior work.

1.4 Structure of the thesis

This thesis is organized into seven chapters. Chapter 2 presents the literature review covering HEMS concepts, LLM-based agents for energy management, the ReAct framework, multi-agent LLM systems, retrieval and solver tools for agents, and the Finnish energy market context. Chapter 3 describes the Design Science Research methodology and evaluation strategy. Chapter 4 presents the system design and implementation. Chapter 5 presents the results on an exploratory and a held-out benchmark of 72 scenarios each. Chapter 6 discusses findings, limitations, and future work. Chapter 7 concludes the thesis.

2. Literature Review

This chapter reviews the background and prior empirical work on which the design and evaluation of the multi-agent system rest. It covers six topics and ends with the research gap that the research questions address. The studies of language models in household energy management were found through a structured search of Scopus and IEEE Xplore, described in Appendix A.

2.1 Home energy management systems

A Home Energy Management System (HEMS) is a software or hardware system that monitors, controls, and optimizes residential electricity consumption to reduce cost, support grid stability, and improve energy efficiency (Beaudin & Zareipour, 2015). The core scheduling problem is the assignment of flexible appliance loads to time windows where electricity prices are lowest, while ensuring that total household load at any moment does not exceed safe operational limits. A load is flexible when its start time can be shifted without violating user preferences or physical constraints.

HEMS scheduling approaches fall into three broad groups. Rule-based systems apply expert-defined heuristics, for example starting an appliance when the price falls below a threshold or in the cheapest hours of a window. They are simple to run and easy to follow, because each decision follows from one stated rule, but each rule covers only the situation it was written for. A rule that picks the cheapest hours for one device does not see the other devices, so it cannot keep the household under a limit on its total power, and a new requirement means writing and testing a new rule.

Mathematical optimization methods such as Mixed Integer Linear Programming (MILP) compute the best schedule for a precisely specified problem. Sou et al. (2011) formulate appliance scheduling with binary start variables, uninterrupted cycles and time windows, and Moradzadeh and Tomsovic (2013) formulate residential energy management as a two-stage optimization that respects the operational constraints of the distribution network. Such methods are optimal for the model they solve, but every constraint must first be expressed in mathematical form, so a household's preferences have to be translated into variables and constraints by someone who can do so, and a new kind of constraint needs a new formulation. Their output is a schedule, not a reason for it.

Learning-based methods acquire a scheduling policy from data, as in the deep reinforcement learning approach of Mocanu et al. (2019), which is trained on measured building data and then optimizes schedules on-line. They adapt to the household they are trained on, but a change in the task, such as a new appliance or a new tariff, generally requires new training, and the learned policy is hard to inspect.

Optimization and learning-based controllers have been described as black boxes that are hard for users to understand and trust, which has led to work on explainable HEMS, for example by distilling a reinforcement-learning policy into a decision tree that can be explained to end-users (Gokhale et al., 2024). Zharova et al. (2024) argue that users need to understand why and how a recommendation system reaches its recommendations before they trust and accept them, and add a dedicated explainability agent to a multi-agent system that recommends when to run household appliances. Language models have also been used to supply the explanation: Zhang and Chen (2024) combine Shapley values with a language model that turns the feature contributions of a machine learning controller into a narrative, and test the approach on model predictive precooling during demand response events in a virtual testbed. Kovacs et al. (2025) add textual explanations generated by a language model to a recommender that suggests when to run energy-intensive household activities during solar production peaks; in a questionnaire with 27 respondents, a hybrid of text and visual explanation was preferred. Explanation in terms a non-expert understands, and adaptation to new constraint types without a new rule, formulation or training run, are therefore the two properties that a language model agent could add to these approaches.

Rule-based and optimization approaches are both available to households today as open-source tools. Among Home Assistant users in Finland and the other Nordic countries, a widely shared approach is to automate devices with community blueprints that switch a device on during the cheapest hours of a user-defined time window, based on Nord Pool day-ahead prices (Kert, 2023). A related tool, the Home Assistant integration Nordpool Planner (Daniel, 2022), chooses the block of consecutive hours with the lowest average price before a set end time. Such rules handle each device separately and do not consider the household's total load. EMHASS (Energy Management for Home Assistant) is an open-source optimizer that schedules deferrable loads, batteries and photovoltaic production against price forecasts with linear and mixed-integer programming (EMHASS contributors, 2026). It supports a time window for each load, a limit on the power drawn from the grid, and loads defined by a fixed power profile. Both kinds of tool require the household to describe each appliance in structured form, namely its power, run time and time window. OpenHEMS, a Home Assistant add-on, schedules heavy appliances under a maximum power but targets fixed off-peak tariffs rather than spot prices (OpenHEMS contributors, 2026). This thesis uses the cheapest-hours rules and EMHASS as reference points for the agents (Section 3.3).

2.2 LLM-based agents for energy management

Studies applying LLMs to energy management have appeared since 2023, and two recent ones are close to this thesis. El Makroum, Zwickl-Bernhard, and Kranzl (2026) present a hierarchical agentic architecture for home energy management in which an orchestrator agent delegates to three specialist agents that follow the ReAct pattern. The household has three flexible loads: a washing machine (2.0 kW for 120 minutes), a dishwasher (1.8 kW for 90 minutes) and an electric vehicle charger (7.4 kW for 360 minutes). The agents receive Austrian day-ahead prices from ENTSO-E for one day, 14 October 2025, and are

run with three open-weight models, Llama 3.3 70B, Qwen 3 32B and GPT-OSS 120B. The evaluation consists of 75 runs: 15 single-appliance runs, 15 multi-appliance runs and 45 runs in which the user asks analytical questions about the schedule. Each schedule is compared with the optimum of a MILP model rather than with an unscheduled baseline, so the study reports how often the agents reach the optimal cost, not how much they save. All three models scheduled single appliances optimally, but only Llama 3.3 70B coordinated all appliances in every multi-appliance scenario, and the analytical questions were answered unreliably unless the agents were given explicit guidance. The authors describe the study as a proof of concept: one household, one day and one price series, without thermal loads such as heat pumps, electric heating and hot water storage. They name pilot deployments and evaluation across electricity markets and household types as future work. Its architecture is close to the pipeline of this thesis, and several of the systems described below share parts of it. The difference lies less in the architecture than in the evaluation: this thesis measures the saving against no scheduling on held-out days, across several price patterns, and against tools that households already use.

Jonah et al. (2026) take the opposite design choice: a single tool-calling ReAct agent that turns a household's request into appliance schedules, using half-hourly dynamic prices from the United Kingdom (Octopus Agile), weather and photovoltaic forecasts and household demand data. They test five language models, among them Claude Sonnet 4.6, GPT-4o-mini and Qwen 3. Their agents capture 96.7 to 98.0 % of the savings available between an off-peak timer and a MILP optimum and do better than rule-based policies. Their failure analysis is the more important result for this thesis. The models sometimes fabricated schedules or failed to commit one, and in some cases a model explained a deadline correctly and still committed a schedule that violated it. Actions parsed from text were less reliable than native function calls. Building on the study of El Makroum, Zwickl-Bernhard, and Kranzl, they list multi-appliance coordination failures, scheduling on price alone, a single-day evaluation horizon and untested constraint safety as open problems, and they recommend that an independent deterministic validator check every schedule against duration, deadline and power constraints before it is executed. Their evaluation covers one household over twelve tariff days and a one-week deployment. Section 2.7 sets out how these open problems shape the design and evaluation of this thesis.

Further studies, mostly conference papers from 2024 to 2026, apply language models to the scheduling itself. Siino and Tinnirello (2024) ask a GPT model to re-time the use of household appliances in a residential consumption dataset so as to reduce cost while keeping each appliance's daily use unchanged, and describe this as a first and preliminary attempt. Yang et al. (2025) propose LLM-MAC, a zero-shot hierarchical framework in which one agent defines constraints, a second optimizes the schedule and a third resolves conflicts iteratively; they report feasible, cost-efficient and interpretable schedules across household configurations. S. Liu et al. (2025) apply a similar zero-shot idea to smart buildings, with zone agents that express local intent, a building agent that reconciles them and an LLM scheduler that makes and explains the final decision. Raghavan and Giridhar (2025) use a small language model, Phi-3 Mini, to interpret commands and call the tools of a model predictive controller that schedules a battery, and report a higher net

profit in a daily scenario than a general controller and a price-blind baseline. Mongaillard et al. (2024) construct three language model agents that turn a voice request into an optimization problem, identify its parameters and solve it, and test the architecture on electric vehicle charging requests with Llama 3 8B. In El Makroum, Zwickl-Bernhard, Kranzl, and Auer (2026), an aggregator agent and a household energy management agent negotiate flexibility requests in natural language, and the household agent assesses deliverability and cost by calling an optimization-based tool; interactions complete in under 12 seconds in a proof of concept. Rashed et al. (2025) use a language model for the load forecasting agent of a five-agent demand response system and report a 14 % reduction in residential consumption, Dolinin and Kiourt (2026) coordinate device agents under an orchestrator and report up to 16 % lower air-conditioning power under critical load, and Moghanjooghi et al. (2025) combine LSTM forecasts of appliance use with GPT-4o to guide users in shifting consumption to off-peak hours.

A second group of studies uses language models on the interface, advice and recommendation side of household energy management and leaves the schedule to another component. Michelon et al. (2025) build an interface in which a language model, using ReAct and few-shot prompting, turns a household's loosely worded answers into the parameters of a home energy management system. With users simulated by another language model, it retrieves 88 % of the parameters correctly on average, more than the same interface without ReAct or few-shot prompting. Oprea and Bâra (2024) use zero-shot classification with a language model to recommend, for a prosumer with a 5.9 kW system, whether to decrease or increase consumption or to sell or buy electricity in two scenarios. A. Papaioannou et al. (2025) disaggregate appliance-level consumption, profile household routines and fine-tune a language model to turn the result into personalized energy-saving advice. I. Papaioannou et al. (2025) fine-tune several language models, among them GPT-4, to generate recommendations from building conditions, forecasts and electricity prices, and report an estimated reduction of 10 % in electricity consumption during a two-week operation in a residential building. These studies measure the accuracy of parameters, labels or advice, or total consumption, and none of them produces a schedule whose cost is compared with an unscheduled day or an optimum.

Assistants that combine a language model with domain tools follow the same pattern. Jung (2026) presents HEMA, an open-source multi-agent assistant with analysis, knowledge and control agents and 36 domain-specific tools, and evaluates it with language-model-simulated users on 23 metrics. Gkalinikis et al. (2025) fine-tune a pre-trained model to generate calls to three external energy analytics tools only when needed, as the core of a home energy adviser. Gonçalves et al. (2025) connect IoT sensor data to a language model that gives personalized recommendations through a conversational interface, and report a 5.66 % reduction in energy consumption against baseline in a 12-month randomized controlled trial with 100 households. Systematic reviews describe the language model in this field mainly as a decision-support and knowledge-integration component: Cali et al. (2026) analyse 106 studies of generative AI in renewable energy and smart grids, and Ghafari et al. (2025) review language models in power systems.

Smart-home assistants show a related pattern without a price signal. Sasha (King et al., 2024) lets a language model plan device actions and automation routines for goals stated without naming devices, such as helping the user save energy, and its user study documents failure patterns that reduce its usefulness. Giudici et al. (2024) present GreenIFTTT, a GPT-4 assistant that helps users create and control home automation routines for energy consumption optimization; an exploratory study with 13 participants found it usable and engaging. Both studies concern how users express intent and how routines are created, not the quality of a multi-appliance schedule.

2.3 The ReAct framework

In the ReAct (Reasoning and Acting) framework of Yao et al. (2023), an LLM agent interleaves reasoning with actions instead of first planning everything and then acting. Each step has three parts: a Thought, which reasons about the current state and the next action; an Action, which is a structured call to an external tool or API; and an Observation, which is the result the tool returns. The loop repeats until the agent reaches a final answer.

Yao et al. (2023) evaluate ReAct on four benchmarks of two kinds. Two are knowledge-intensive reasoning tasks, in which the agent can query a Wikipedia API: HotpotQA, multi-hop question answering, and FEVER, fact verification. Two are interactive decision-making tasks: ALFWorld, a text-based game in which the agent carries out household chores in a simulated home, and WebShop, in which it shops on a simulated website to match a customer's instructions. Their baselines are built by removing parts of the ReAct trajectories: standard prompting without reasoning or actions, chain-of-thought prompting with reasoning but no actions, and acting alone without reasoning, so that each comparison isolates the contribution of reasoning or of acting. With PaLM-540B, ReAct reached 27.4 exact match on HotpotQA, slightly below chain-of-thought (29.4), and 60.9 % accuracy on FEVER, above it (56.3 %), while the best results on both came from combining the two. On the interactive tasks the advantage was clear: on ALFWorld the best ReAct trial reached a 71 % success rate against 45 % for acting alone, and on WebShop 40.0 % against 30.1 %. The gains were largest where the agent has to act on information it looks up during a multi-step task. Household scheduling is such a task: the agent looks up prices, loads and costs while it builds the schedule. The trace also makes the result explainable, because it records the reasoning behind every tool call, so it shows why an appliance was given a particular start time.

2.4 Multi-agent LLM systems

Multi-agent LLM systems have been proposed as a solution to the context window and coordination limitations of single-agent designs. Han et al. (2024) discuss the challenges and open problems of LLM multi-agent systems, among them task allocation, robust reasoning through iterative debate, the management of complex and layered context information, and memory management. Role specialization, in which each agent is

responsible for one well-defined subtask and has access only to the tools that subtask needs, is one response to these challenges and the design principle followed in this thesis.

Hong et al. (2024) demonstrate role-based multi-agent orchestration for software development with MetaGPT, in which agents act as product manager, architect, project manager, engineer and quality-assurance engineer. Encoding standard operating procedures and requiring structured intermediate outputs, exchanged through a shared message pool, reduces the inconsistencies that arise when language model calls are simply chained, and MetaGPT reported state-of-the-art results on code-generation benchmarks at the time. Structured hand-offs between specialists address the same kind of problem as the multi-appliance coordination failures reported for LLM-based HEMS (El Makroum, Zwickl-Bernhard, & Kranzl, 2026).

The move from one agent to a hierarchy of agents is motivated by three problems of a single agent that handles a whole task. First, its context grows with every tool call and observation, and the model has to keep track of all of them, while a specialist sees only the information of its own step. Second, a single agent has access to every tool, so it can call any of them at any point, while a specialist is limited to the few tools its subtask needs, which narrows the choices it can get wrong. Third, the intermediate results of a single agent stay inside its own reasoning, while a hierarchy passes them between agents as structured outputs, which can be checked before the next step (Hong et al., 2024). In a hierarchical design an orchestrator splits the task, hands each part to a specialist and combines the results; the survey of Guo et al. (2024) describes such layered structures alongside centralized and decentralized ones as ways in which the agents of an LLM-based multi-agent system communicate. The hierarchy has a price: more model calls per task, hand-offs that can lose information, and the need to decide in advance how the task is split. For household scheduling the split follows the structure of the problem: which appliances can move, when each is cheapest, and how they fit under the peak limit together. The verification step between the specialists is where the independent check recommended by Jonah et al. (2026) fits.

2.5 Retrieval and solver tools for LLM agents

Two techniques extend what a language model agent can do without retraining it. Retrieval-augmented generation (RAG) gives the model relevant documents from an external store at the time of the request, so that its answer draws on knowledge it was not trained on and the store can be updated independently of the model (Lewis et al., 2020). In smart homes, retrieval has been used to give the model the capabilities of the installed devices, so that a requirement stated in words becomes executable service code (Jahanbakhsh et al., 2025). For a planning task, the retrieved material can be solved examples of similar problems, which show the model what a good solution looks like. Whether such examples improve the search of a multi-step plan, rather than only its form, is an empirical question.

The second technique keeps the search out of the language model altogether. In OptiGuide,

a language model answers questions about a supply chain by writing and running code against an optimization solver and explaining the result; the authors stress that the aim is to use solvers in tandem with language models rather than to replace them (Li et al., 2023). A household scheduling agent can be built the same way: the agents collect the household's constraints, call a solver, check its answer and explain it, while the solver guarantees the best schedule. Solver tools have recently been combined with language models in home energy management: an agent calls a model predictive controller (Raghavan & Giridhar, 2025) or an optimization-based tool to assess flexibility requests (El Makroum, Zwickl-Bernhard, Kranzl, & Auer, 2026), and a deterministic supervisor checks the plan of a language model before a MILP scheduler executes it (Raghavan & Giridhar, 2026). Retrieval of solved examples has not been evaluated for household appliance scheduling in the studies reviewed here, and neither technique has been compared with the home energy management tools that households already use. This thesis studies both as extensions of the pipeline (Section 4.4).

2.6 Finnish energy market context

Finland belongs to the Nordic spot electricity market (Nord Pool) and has priced electricity in 15-minute intervals since October 2025, which makes its residential market one of the most finely priced in Europe. At the end of 2024, about 33 % of all Finnish electricity contracts were dynamic-price contracts, up from 9 % in 2021; these pass hourly and quarter-hourly price changes straight to the consumer (Energy Authority, 2025).

Prices within a day vary a great deal. In the Nord Pool FI day-ahead prices from 2023 to 2026, the difference between the cheapest and the most expensive hour of a day had a median of 5.7 c/kWh excluding VAT and exceeded 10 c/kWh on 28 % of days. On days without near-zero prices the most expensive hour cost a median of 3.1 times the cheapest, and on 19 % of days at least one hour had a negative price (computed from Pakastin Oy, 2026). Extremes are far larger: the highest hourly price in the period, 189.6 c/kWh on 5 January 2024 at 19:00, was about one hundred times the median night-time price of 1.8 c/kWh. On 24 November 2023 the price fell to -500 EUR/MWh for half a day, and households with spot-price contracts noticeably increased their consumption (Haakana et al., 2025). At the same time, an estimated 7 to 15 % of Finnish households are energy poor (Aalto University, 2026). Shifting household demand in Finland therefore matters for the power system as well as for household bills.

2.7 Literature synthesis and research gap

Three findings come out of the reviewed literature. First, HEMS scheduling is a multi-step, multi-constraint optimization problem. Rules and optimizers solve it for the cases they were written or formulated for but do not explain their decisions, and language model agents have been proposed because they can take constraints stated in words and explain what they did (Section 2.1). Second, the first LLM-based HEMS

implementations, both hierarchical and single-agent, have documented limitations in multi-appliance coordination, in scheduling on price alone, in the safety of committed schedules, and in evaluation scope (El Makroum, Zwickl-Bernhard, & Kranzl, 2026; Jonah et al., 2026). Other studies use language models for the interface, advice or recommendation side of household energy management, without producing a schedule whose cost is compared with a baseline or an optimum (Michelon et al., 2025; Oprea & Bâra, 2024; A. Papaioannou et al., 2025; I. Papaioannou et al., 2025). Third, the existing home energy management tools that households can install range from cheapest-hours rules that plan each device alone to optimizers such as EMHASS, and retrieval of solved examples and solver tools are established ways to strengthen language model agents (Lewis et al., 2020; Li et al., 2023). None of the HEMS studies reviewed here measures what an agent adds over these existing tools across several price patterns, or whether retrieval or a solver tool lets an agent reach the saving of an optimizer. Table 2.1 links each limitation of the prior work to the design or evaluation choice of this thesis that responds to it, and to the research question that tests it.

This thesis addresses that gap with three contributions. The first is the Multi-Agent Pipeline, a hierarchical ReAct multi-agent system in LangGraph that gives households daily appliance recommendations: two agents inside a deterministic harness that profiles the appliances, retrieves solved days and checks every schedule, with an optimiser tool. The design iterations that led to it are documented, and the pipeline is compared with a single agent in the same harness. The second is a measure of what its recommendations save, how often they are valid and what they cost to produce compared with a language model answering without tools (SQ1). The third is a map of the conditions, four price patterns and three household loads under the 8 kW power charge threshold, under which the recommendations save more than, or as much as, existing home energy management tools, cheapest-hours automations and EMHASS (SQ2). The hypotheses were fixed in advance and tested on a held-out benchmark.

Table 2.1 Limitations of prior work and how this thesis responds to them.

Limitation in prior work	Response in this thesis	Tested in
Multi-appliance coordination fails for some models (El Makroum, Zwickl-Bernhard, & Kranzl, 2026; Jonah et al., 2026)	An optimiser tool, retrieval of solved days, conflict tools for repairs, and an electrified household load (car charge and sauna) that forces appliances to be staggered under the 8 kW limit	SQ1, SQ2
Committed schedules can violate constraints the model reasoned about correctly; an independent check is recommended (Jonah et al., 2026)	A deterministic verification step checks every schedule for conflicts, missing appliances and window violations, and returns it for revision	SQ1 (validity)
Evaluation on a single day or a few days, one household, one price series (El Makroum, Zwickl-Bernhard, & Kranzl, 2026; Jonah et al., 2026)	A factorial benchmark of four price patterns and three household loads, built from 18 REFIT households, and a held-out benchmark with pre-registered hypotheses	SQ2
Scheduling on the price signal alone (Jonah et al., 2026)	Partly addressed: every schedule must also keep base load plus appliances under the 8 kW threshold of the Finnish power charge; the objective remains the electricity cost, and other signals such as carbon intensity are not added	SQ2
No comparison with existing home energy management tools	Comparison with cheapest-hours automations for Home Assistant and with the optimizer EMHASS	SQ2
Rules and optimizers do not explain their decisions (Gokhale et al., 2024; Zharova et al., 2024)	The Orchestrator explains every recommendation and states the result of the final peak-load check	design

3. Research Methodology

This chapter describes the research methodology and explains how Design Science Research is applied in this thesis. It presents the rationale for the approach and the mapping of its phases to the thesis, the theories on which the design of the artifact draws, and the evaluation strategy: the systems compared, how the data were collected, and how they were analysed.

3.1 Design Science Research methodology

This thesis follows the Design Science Research (DSR) methodology as defined by Peffers et al. (2007). DSR designs and evaluates artifacts that solve practical problems. As Hevner et al. (2004) argue, design science creates and evaluates IT artifacts for identified organizational problems and produces knowledge by building and testing them. The main activity of this thesis is building and evaluating the multi-agent system, so DSR fits it.

Artifacts in DSR are classified into four types: constructs, models, methods, and instantiations (March & Smith, 1995). In this thesis, the artifact is an instantiation, that is, a working software implementation of a hierarchical multi-agent scheduling pipeline. It is supported by architectural models (the LangGraph state graph design) and constructs (the division into language model agents, Orchestrator and Scheduler, and deterministic harness steps for profiling, retrieval and verification). Table 3.1 maps the six DSR phases defined by Peffers et al. (2007) to the activities and outputs of this thesis.

Table 3.1 DSR phase mapping.

Phase	DSR step	Activity in this thesis	Output
1	Problem identification	Structured literature search (Appendix A); in the studies reviewed, no LLM-based HEMS is evaluated across several price patterns against tools that households already use	Gap statement; confirmed RQs
2	Define objectives	Specify measurable success criteria: saving against no scheduling, validity, conflict resolution rate, latency, token use	Objective specification
3	Design and development	Implement the Orchestrator and the Price-Aware Scheduler in LangGraph inside a deterministic harness (profiling, retrieval of solved days, verification) with an optimiser tool; build a single agent in the same harness for comparison	Working multi-agent artifact
4	Demonstration	Run pipeline on five representative scenarios; confirm valid cost-reducing schedules	Demonstration results
5	Evaluation	Exploratory runs on the factorial benchmark (72 hourly-price scenarios); a pre-registered confirmatory run on 72 held-out fifteen-minute scenarios (Q3); comparison with existing home energy management tools and with a single agent; Wilcoxon, TOST and Mann–Whitney tests	Quantitative evaluation results
6	Communication	Write findings, design guidelines, limitations, and future work	Complete thesis

3.2 Theoretical foundations

Design science research grounds the design of an artifact in kernel theories, the existing theories from which its design principles are derived, and positions its contribution against them (Gregor & Hevner, 2013). Four bodies of work serve this role here. The first is demand flexibility under time-varying prices: a household lowers its cost by moving flexible loads to low-price periods, within the time windows its members accept and the power limits of its connection. This is the problem that rule-based and optimization-based HEMS address (Section 2.1), and the MILP formulation of Sou et al. (2011) states it precisely. It defines what a good recommendation is: the cheapest schedule that respects every window and the peak limit.

The second is reasoning grounded in tool observations. ReAct (Yao et al., 2023) holds that a language model solves multi-step tasks more reliably when it alternates between reasoning and actions whose results it observes, instead of producing an answer from its parameters alone. It yields the design principle that every number an agent uses comes

from a deterministic tool, and the LLM without tools is the baseline that tests it (H1a). The third is task decomposition with role specialization in multi-agent systems (Hong et al., 2024; Guo et al., 2024): a task split into specialist roles with structured hand-offs keeps each agent's context and tool set small and makes intermediate results checkable (Section 2.4). It yields the four roles of the pipeline and the verification step between them.

The fourth is the reuse of solved problems. Case-based reasoning solves a new problem by retrieving similar past cases and adapting their solutions (Aamodt & Plaza, 1994), and retrieval-augmented generation supplies a language model with such material at the time of the request (Lewis et al., 2020). Together with the use of a solver as a tool (Li et al., 2023), it yields the case library and the optimiser tool of the final design. Guidelines for human-AI interaction (Amershi et al., 2019) add the requirement that the system explain what it did and state when a constraint is not met, which the Orchestrator's summary implements.

The evaluation follows the Framework for Evaluation in Design Science Research (FEDS) of Venable et al. (2016). All evaluations are artificial, on simulated household days, and they move from formative to summative. The design iterations were formative evaluations on an exploratory benchmark, and the pre-registered run on a held-out benchmark is the summative evaluation. Of the strategies FEDS describes, this corresponds to Technical Risk and Efficacy, which is suited to an artifact whose main risk is whether it works technically before it is evaluated with users. A naturalistic evaluation with real households is left to future work (Section 6.5).

3.3 Evaluation strategy

The evaluation is a controlled simulation that needs no sensors or hardware. Its inputs come from public datasets and APIs, and every scenario is built from measured appliance data and historical prices. The comparison is paired: every system receives the same scenarios and is scored by the same function.

Research questions. The evaluation answers the research question and the two sub-questions posed in Chapter 1, which are restated here because the rest of this section is organised around them:

> **RQ:** How can a hierarchical ReAct multi-agent system (the Multi-Agent Pipeline) be designed to give households reliable, cost-saving daily appliance recommendations under dynamic electricity prices?
>
> **SQ1 (value and cost):** How much do the pipeline's recommendations save, how often are they valid, and what do they cost to produce, compared with a language model that answers without tools?
>
> **SQ2 (conditions):** Under which price patterns and household loads do the

pipeline's recommendations save at least as much as existing home energy management tools?

The RQ is answered by the design itself, by the evidence that the design meets its objectives (Section 4.1), and by a comparison of the pipeline with a single agent in the same harness, which shows what the division into two agents adds. SQ1 and SQ2 are answered by measurement: each is tested by the registered hypotheses of Table 3.2, H1a to H1c for SQ1 and H2a to H2e for SQ2, and SQ2 is also examined descriptively for each price pattern and household load. The systems that SQ1 and SQ2 compare are defined next.

Systems compared. The artifact is the *Multi-Agent Pipeline* with GPT-6 Luna (Chapter 4): an Orchestrator and a Price-Aware Scheduler inside a deterministic harness. The Scheduler receives the two most similar solved days from a case library as worked examples and has a tool that solves the whole day exactly; it decides when to call it, and the harness checks the answer before the Orchestrator explains it (Section 4.4). Its earlier versions are design iterations: they were run on the exploratory benchmarks and are reported in Section 5.3.1, but they are not compared systems. The *single agent in the harness*, one ReAct agent with the Scheduler's tools, the same context and the same checks, is reported without hypotheses as part of the answer to the RQ. The pipeline is compared with the following alternatives:

- *No scheduling*: every appliance starts as soon as the user allows. It defines the saving of every other system.
- *Cheapest-hours automation*: the selection rule of the "Nordpool cheapest hours actions" blueprint for Home Assistant (Kert, 2023). Within the appliance's window, the prices are sorted in ascending order and the threshold is the price at index $4h$ (counting from zero), where h is the cycle length rounded up to whole hours. The blueprint switches a device on whenever the price is at or below the threshold; for a cycle appliance, it is assumed that this trigger starts the cycle, which then runs to completion.
- *Cheapest window*: the block of whole hours, as long as the cycle rounded up, with the lowest average price between the ready time and the deadline. This is how the Home Assistant integration Nordpool Planner (Daniel, 2022) chooses when to switch a device on.
- *Smart timer (own heuristic)*: a peak-aware rule written for this study as a reference point, not a deployed product. It shows how far a simple rule gets without a language model or optimisation, and a household energy application could implement it. It places the appliances one at a time, the most energy-intensive first, each at its cheapest start at which the household load stays within the peak limit. The rule was fixed before it was run on the benchmark and was not tuned afterwards.
- *EMHASS* 0.18.4, the open-source optimizer Energy Management for Home Assistant (EMHASS contributors, 2026), run as households run it: as a Docker

service with its default settings, called through its REST interface. Each scenario is sent as runtime parameters of a day-ahead optimisation: the prices, the base load as the load forecast, no photovoltaic production, the appliances as deferrable loads with their time windows and measured power profiles, and the peak limit as the maximum power drawn from the grid. Home Assistant itself is not involved, since all data is passed with the request. A configuration with a constant nominal power per appliance, set to the cycle's peak, is reported as a secondary result. As a cross-check, the same scenarios were also solved by calling EMHASS's optimisation module directly with an exact solver tolerance; the two routes give the same schedule in most scenarios, and the remaining differences stem from the default tolerance of the service.

- *LLM without tools*: the same model as the agents, given the day's appliance plan together with all household data and the price and measured base load of every slot in a single prompt. Its schedule is taken from one answer, without tools or a reasoning loop. This corresponds to the standard prompting baseline of Yao et al. (2023) and isolates the contribution of tool use.

The cheapest-hours rules plan each appliance on its own and ignore the household's total load, exactly as the real tools do. Because language model output varies between runs, each system that uses a language model is run three times per scenario, while the deterministic systems are run once.

The MILP solver of Section 4.5 is not a compared system. It confirms during benchmark construction that every scenario has a feasible schedule, it solves the days of the case library, and it is the solver behind the optimiser tool. Since a schedule cannot save more than the MILP schedule, its saving is reported as the upper bound in the result tables.

Stages. The system was developed over the design iterations of Section 5.3.1, first on a benchmark of 48 scenarios with hourly prices in six scenario types and then on factorial exploratory benchmarks in which the price pattern and the household load vary separately (Section 5.1.4), because each of the old types changed several properties at once. Along the way the saving against no scheduling replaced the gap to the optimum as the primary outcome, retrieval and the optimiser tool were added, the four agents of the earlier design were reduced to two agents in a deterministic harness, and the peak limit was set to the sourced 8 kW threshold, with an electrified household load so that the appliances still compete for capacity (Section 5.1). The hypotheses of Table 3.2 were then fixed in a pre-registration, committed to the project repository together with its analysis script before any run on the held-out set Q3 of 72 scenarios with fifteen-minute prices, whose days appear in no exploratory benchmark and not in the case library. Only the run on Q3 is used to accept or reject them.

Table 3.2 Hypotheses and statistical tests (pre-registration, held-out set Q3).

	Hypothesis	Test
SQ1, value and cost (all 72 scenarios)		
H1a	The Multi-Agent Pipeline saves more than the LLM without tools.	Wilcoxon signed-rank
H1b	Its recommendations are valid more often than those of the LLM without tools.	Wilcoxon signed-rank
H1c	Its mean response time is below 60 s and its mean API cost below \$0.10 per recommendation (two tests).	One-sided t test against the limit
SQ2, conditions (all 72 scenarios; H2d and H2e: 24 electrified against 48 other)		
H2a	The Multi-Agent Pipeline saves more than the cheapest window.	Wilcoxon signed-rank
H2b	It saves more than the cheapest-hours automation.	Wilcoxon signed-rank
H2c	It saves the same as EMHASS within ± 1 percentage point.	Paired t TOST
H2d	Its advantage over the cheapest-hours automation is larger on electrified days than on the other days.	Mann–Whitney U
H2e	Its advantage over the cheapest window is larger on electrified days than on the other days.	Mann–Whitney U

3.3.1 Data collection

The study uses secondary data only; no new measurements were taken and no human participants were involved. Two kinds of input data are used. Appliance cycles, base loads and fixed appliances come from the REFIT Electrical Load Measurements dataset, which records appliance-level power in 20 UK households at eight-second resolution between 2013 and 2015 and is published under an open licence with the households anonymised (Murray et al., 2017). Day-ahead electricity prices for the Nord Pool FI bidding zone come from two public services that republish Nord Pool data, Sähkötin and Elering, and were checked against the official prices of the ENTSO-E Transparency Platform (Section 5.1). Both are combined into benchmark scenarios by a fixed, seeded procedure, described in Section 5.1, so that every benchmark can be rebuilt exactly.

The evaluation data are generated by running the systems on these scenarios. Every run is stored as one record in a results file. The record contains the schedule, its cost, whether it is valid and which constraint it violates if not, the number of input and output tokens, the API cost, the response time, the number of tool calls and feedback rounds, the number of recovery nudges, and any error the agents reported. The evaluation runs were made with LangSmith tracing switched off, so that sending traces would not add to the measured response time; reasoning traces were recorded in LangSmith during development. EMHASS is called through its REST interface, and its schedules are scored by the same function as all other systems. Runs are resumable, so an interruption repeats only the run in progress, and a run that failed because of a provider or network error is retried and not recorded. Every completed run is kept, including invalid ones.

3.3.2 Data analysis

Scoring

All schedules are scored by the same function. The cost of a schedule is the consumer price of electricity multiplied by the energy of the flexible appliance cycles, summed over the day; the consumer price adds the retail margin, distribution fee, electricity tax and VAT to the spot price (Section 5.1). The base load costs the same under every schedule and is therefore excluded from the cost, but it counts towards the peak load limit. The peak limit is the highest total power the household may draw: in every fifteen-minute slot, the base load plus the power of the appliances running in that slot must stay at or below it. The limit is 8 kW, the threshold of the Finnish power charge, and it is treated as a hard constraint rather than a priced one (Section 5.1). A schedule is valid when every appliance has a start time, every cycle runs within its user window, and the total household load never exceeds the peak limit. An invalid schedule, or a run that returns no schedule, is scored at the cost of no scheduling, on the reasoning that the household would then fall back to starting its appliances when they are ready. If starting every appliance when it is ready exceeded the limit itself, it would nevertheless remain the reference cost, because it is what the household would do without a recommendation, so an invalid schedule saves nothing against it. The validity rate is reported separately, so that this convention does not hide failures.

The primary outcome is the saving against no scheduling:

$$\text{saving}(\%) = \frac{\text{cost without scheduling} - \text{system cost}}{\text{cost without scheduling}} \times 100.$$

It is what a household gains by following the recommendation, and it needs no optimum as a reference. Because the consumer price stays positive even when the spot price is negative, the cost without scheduling is always positive. Over a set of runs, the pooled saving divides the summed savings by the summed costs without scheduling, so that expensive days weigh more than cheap ones, as on a bill; the paired tests use the per-scenario saving. An invalid run saves nothing. The gap to the MILP optimum, $(\text{system cost} - \text{MILP cost})/\text{MILP cost} \times 100$, is reported as a secondary measure, because it shows how much saving a system leaves on the table.

The choice of tariff does not change the ranking of the systems. The margin, distribution fee and electricity tax are constant per kilowatt-hour, and every cycle uses the same energy wherever it is placed, so the consumer price changes the cost of every schedule by the same amount and scales all cost differences by the common factor $1 + v$. The ranking of the systems and the euro differences between them are therefore the same as under the spot price, apart from that factor, which was also verified on the recorded schedules. The saving in percent does depend on the tariff, through its denominator: under the spot price alone, the same euro saving would be a larger share. The consumer price is used because it is what the household pays.

The sub-questions use these measures as follows. SQ1 compares the saving and validity of the pipeline with those of the LLM without tools, and checks its response time and API cost per recommendation against the design objectives of Section 4.1 (60 s and \$0.10); tokens (input plus output) are reported alongside. SQ2 compares the saving and validity of the pipeline with those of the existing home energy management tools, the cheapest-hours rules and EMHASS, first over all scenarios and then separately for each price pattern and household load. Whether the advantage over the cheapest-hours rule and the cheapest window grows with the electrified load is tested (H2d, H2e); the rest of the breakdown by condition is descriptive. Also reported without hypotheses are the single agent in the harness, the total euro saving, the smart timer (own heuristic), the gap to the optimum, and the conflict resolution rate, which is the share of valid runs among the scenarios that require coordination (those in which starting every appliance at its individually cheapest time would exceed the peak limit).

Statistical analysis

Two systems are compared per scenario, using the mean saving over the runs as the paired observation. The Wilcoxon signed-rank test assesses whether the difference across the 72 paired scenarios is statistically significant. It is designed for paired comparisons and does not assume that the differences are normally distributed. Pratt's treatment of zero differences is used, so that scenarios in which both systems reach the same cost remain in the ranking instead of being discarded. Practical significance is reported with the effect size $r = |Z|/\sqrt{N}$. All tests are two-sided at $\alpha = 0.05$. A directional hypothesis is supported when its test is significant and the effect has the predicted direction. Two-sided tests are the conservative choice for directional hypotheses, since a one-sided test would halve each p-value. The p-values of the registered hypotheses are adjusted for multiple testing with the Holm procedure within each sub-question: four tests for SQ1 and five for SQ2. The two tests of H1c compare a mean with a fixed limit, which is a one-sided claim by nature, so they are one-sided: the upper end of the one-sided 95 % confidence interval must lie below the limit. H2d and H2e compare two groups of scenarios rather than two systems: the per-scenario saving advantage of the pipeline over a rule on the 24 electrified days against that on the 48 other days, with the Mann–Whitney U test and the rank-biserial correlation as effect size. A Wilcoxon test whose differences are all zero, which happens when two systems reach the same cost on every scenario, cannot be computed and counts as showing no difference.

A non-significant difference does not by itself show that two systems perform alike, since it can also result from too little data. Equivalence with EMHASS (H2c) is therefore tested with the two one-sided tests (TOST) procedure (Schuirmann, 1987; Lakens, 2017). The systems are considered equivalent when both one-sided tests reject, at the 5 % level, a mean difference in saving of at least one margin in either direction. This is the same as requiring the 90 % confidence interval of the mean paired difference to lie inside the margin. The margin is ± 1 percentage point of the cost without scheduling. On the Q3 set that cost has a median of € 0.90 per day (interquartile range € 0.37 to € 1.66, maximum € 4.49), so one percentage point is about 0.9 cents on the median day and under five cents

on the most expensive one, which a household would not notice on its bill. The paired t version of the test concerns the mean difference and is the primary one; a Wilcoxon version, which concerns the typical scenario, and margins of 0.5 and 2 percentage points are reported alongside. If the pipeline and EMHASS reach the same cost on every scenario, the confidence interval is $[0, 0]$, inside the margin, and equivalence holds, although the t statistic is then undefined.

4. System Design and Implementation

This chapter describes the design and implementation of the hierarchical ReAct multi-agent system, the Multi-Agent Pipeline. It presents the problem definition and design objectives, the architecture of the pipeline and of the deterministic harness around its agents, the specifications of the agents and of the single agent built for comparison, the retrieval of solved days and the optimiser tool, the MILP solver, and the technology stack used throughout.

4.1 Problem definition and objectives

Based on the literature synthesis in Section 2.7 and DSR Phase 2, the artifact must satisfy five design objectives. First, multi-appliance coordination: the pipeline must correctly schedule two or more flexible appliances simultaneously without producing overlapping high-load windows. Second, price-volatility robustness: the pipeline must produce cost-reducing schedules under every price pattern and household load of the benchmark. Third, constraint compliance: every schedule must respect user-defined completion deadlines and household load limits. Fourth, interpretability: the full reasoning trace of a scheduling decision must be available for inspection, through LangSmith tracing. Fifth, computational efficiency: the pipeline must complete scheduling within 60 seconds and under $0.10 API cost per scenario using a small commercial language model (GPT-6 Luna; Section 4.6).

4.2 Architecture overview

The artifact is a LangGraph state graph whose nodes are either ReAct agents, which use a language model, or deterministic steps, which do not. All nodes read from and write to a shared state object that persists for the whole run (Table 4.1). Two agents do the work that needs a language model. The Orchestrator turns the household's appliance plan into a task brief listing the appliances to schedule and, at the end, explains the final schedule to the household. The Price-Aware Scheduler plans a start time for every flexible appliance with its tools, among them the optimiser tool (Section 4.4). The deterministic steps form a harness around the agents: they prepare what an agent needs and check what it returns, so that no number and no check depends on the language model.

After the Orchestrator's brief, the profiling step builds the appliance profiles (type, peak power, cycle length and time window) directly from the household data, and the retrieval step adds the two most similar solved days as worked examples to the Scheduler's message. After the Scheduler, the verification step checks its schedule for three kinds of problems: requested appliances without a start time, start times outside the user's time window, and

peak load conflicts. If a problem remains, the step sends a description of it back to the Scheduler, which revises its schedule; the Scheduler runs at most twice, that is, with one feedback round. If an appliance still has no start time after that round, the verification step places it at its cheapest start that keeps the load within the limit. The Orchestrator then writes a short summary for the household that states the result of the final peak load check. The original request travels in the shared state together with the Orchestrator's brief, so an appliance missing from the brief is still profiled and scheduled. Figure 4.1 shows the architecture, and Table 4.1 lists the fields of the shared state.

The earlier design had four agents. An Appliance Profiler classified the appliances with two data tools, and a Conflict Resolver checked the Scheduler's proposal against the peak limit and staggered overlapping cycles. In the design iterations the profiling turned out to be a fixed reading of the household data and, once the Scheduler had the optimiser tool, the Resolver's repairs were almost never needed, while each agent added model calls, tokens and response time. The final design therefore does the profiling deterministically and gives the conflict tools to the Scheduler for the feedback round (Section 5.3.1).

Table 4.1 Fields of the shared LangGraph state.

Field	Written by	Content
request	caller	the household's scheduling request in natural language
brief	Orchestrator	appliance identifiers to schedule and stated constraints
profile	profiling step	per appliance: type, flexible or fixed, peak power, cycle duration, earliest start and deadline
proposal	Scheduler	start time per appliance
checked schedule	verification step	the Scheduler's schedule as checked; in the last round, an appliance still missing is added at its cheapest start within the limit
verification feedback	verification step	appliances without a start time, start times outside the user window and remaining peak conflicts; empty when the schedule passes
rounds	verification step	number of Scheduler rounds completed (at most two)
final, summary	Orchestrator	final start times and the summary shown to the household
tool calls, recoveries, errors	all nodes	counters and error messages recorded for the evaluation

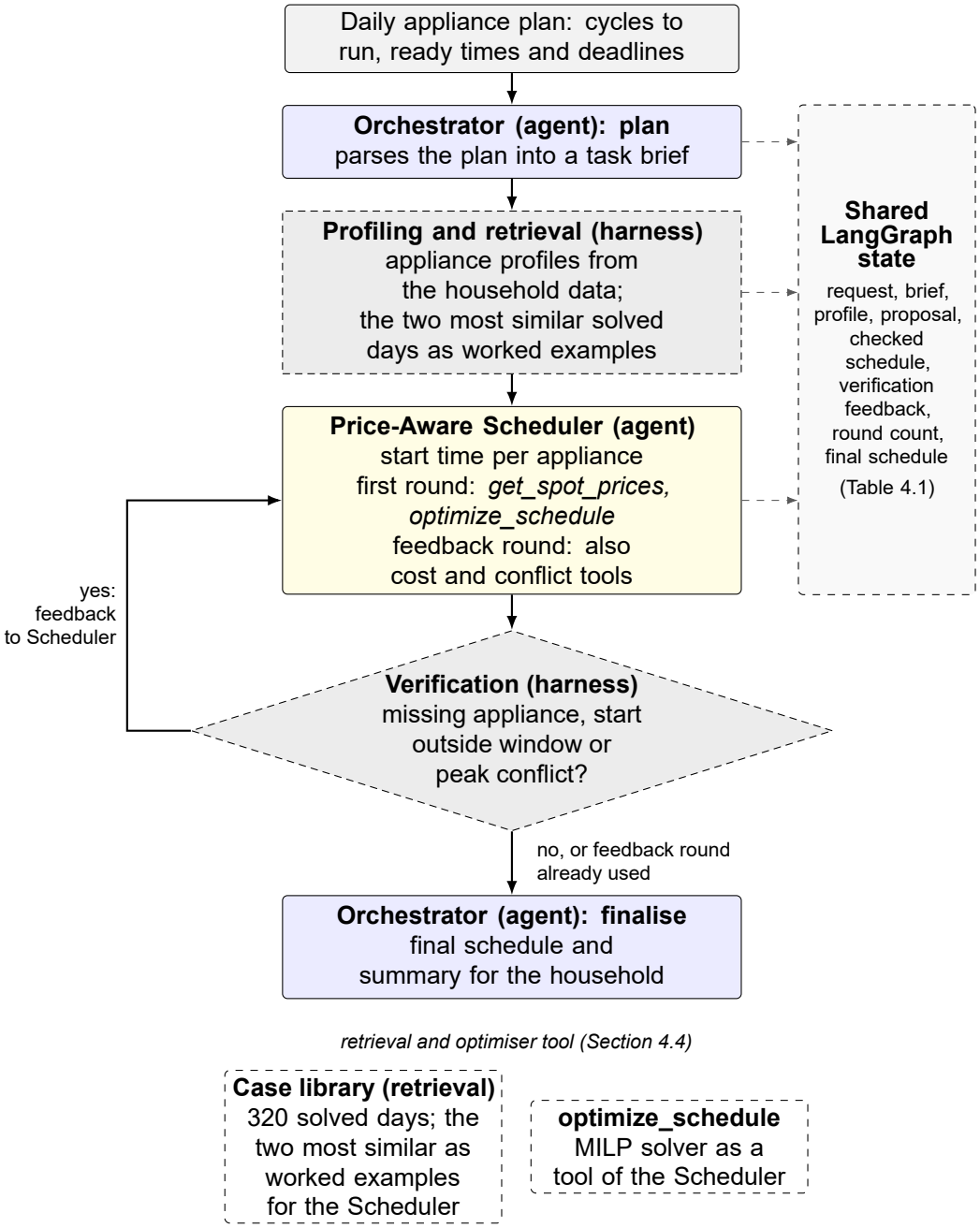


Figure 4.1. Architecture of the hierarchical ReAct multi-agent system: two language model agents (coloured, rounded) and the deterministic harness steps (grey, dashed).

4.3 Agent specifications

The **Orchestrator** reads the household's appliance plan and returns a task brief: the identifiers of all appliances to schedule and any constraint the household states. It has no tools and does no scheduling, so its context stays small; its budget is six graph steps. At the end it receives the final schedule, its cost and the result of the final peak load check

and writes a summary of at most four sentences for the household. It is told never to claim that the constraints are met unless the check confirms it.

The **Price-Aware Scheduler** receives the appliance profiles, one line per flexible appliance, and the worked examples of two similar solved days. In its first response it makes two tool calls together: `get_spot_prices` without arguments, for a summary of the day's prices, and `optimize_schedule` with all flexible appliances. It returns the solver's start times without checking them again, because the verification step checks every schedule after it. If the solver finds no schedule, the Scheduler returns every appliance at its earliest start, and the verification step sends the schedule back with its problems. In that feedback round, and only then, the Scheduler is also given the cost and conflict tools (Table 4.2), so that it can call the optimiser again or move single appliances and confirm that no conflict remains. Offering only two tools in the first round keeps every model call short, since the description of every offered tool is sent with each call. Its budget is 60 graph steps.

Both agents are ReAct agents that must end with a structured output (the task brief or the start time of every appliance), which is validated against a schema before it is written to the shared state. When a model replies in prose instead of producing the required output, or uses up its steps without one, it is given its conversation so far and asked once more to return the output, at most twice. This rule applies identically to every agent and model, and every such recovery is recorded. In the evaluation runs the household summary is not written: the schedule is the recommendation that is scored, and the summary would add one model call to the measured time and cost. The demonstration in Section 5.3 shows it.

The single agent in the harness. To test whether the division into two agents matters, the same harness also runs a single ReAct agent (Figure 4.2). It receives the household's appliance plan itself, without an Orchestrator's brief, together with the same appliance profiles and worked examples. It has the Scheduler's tools, offered in the same two stages, and its schedule is checked by the same verification step. Verification feedback is added to the same conversation, so the agent revises its schedule with its whole history in view, and after the check it writes the household summary in that conversation as well. Its budget per round is the sum of the two agents' budgets, 66 graph steps. Everything that differs from the pipeline is therefore the number of agents and what each one sees.

Table 4.2 lists the tools. They are deterministic functions over the scenario data, so every number an agent reasons with comes from a calculation rather than from the language model. All but one handle part of the problem: `find_cheapest_window`, for example, considers one appliance at a time and ignores both the other appliances and the peak load limit. With these tools alone, coordinating the appliances is left to the agent. `optimize_schedule` solves the whole day (Section 4.4). The profiling step reads the same household data as the data tools of the earlier design, which the agents no longer need.

Before the optimiser tool was added, the automatic mode of `adjust_schedule` was the agents' only way to repair a conflict. It optimizes the start of one appliance while all others

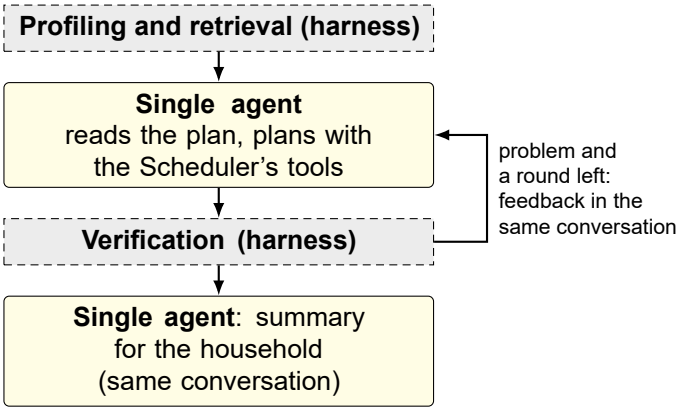


Figure 4.2. The single agent in the same harness, built to compare one agent with two.

Table 4.2 Tools of the Scheduler (and of the single agent), and the round in which each is offered.

Tool (inputs)	Offered	Output and logic
`get_spot_prices(from_time, to_time)`	both rounds	Without arguments, a summary of the day: hourly mean consumer prices and the cheapest and dearest 15-minute slots. With a time range, the consumer price in c/kWh of every 15-minute slot in it. Answers are cached, so a repeated call costs nothing.
`optimize_schedule(appliance_ids)`	both rounds	Solves the day for the given appliances with the MILP of Section 4.5 and returns the start times, the cost and the maximum load.
`calculate_schedule_cost(appliance_id, start_time)`	feedback	Cost of one cycle started at the given time (power profile $\times$ price $\times$ 0.25 h, summed over the cycle), its end time, and whether it lies within the user window.
`find_cheapest_window(appliance_id, top_k)`	feedback	The k cheapest start times within the user window for one appliance, found by evaluating every valid start.
`sum_load(schedule)`	feedback	Maximum total household load (base load plus scheduled cycles) and the five highest slots, compared with the limit.
`detect_conflicts(schedule)`	feedback	Time ranges in which the total load exceeds the limit and the appliances running then, or confirmation that there is no conflict.
`adjust_schedule(schedule, appliance_id, new_start_time)`	feedback	Moves one appliance and reports the cost change, the window check and any remaining overload. Without a start time, it moves the appliance to its cheapest start at which it causes no overload, keeping the others fixed, and lists alternatives.

stay where they are and never re-plans several appliances together. Which appliances to move, in which order, and when to stop were decisions of the agent, and a poor sequence of single moves could settle on a staggered schedule far from the optimum that no single further move could improve. This limitation led to the retrieval of solved days and the optimiser tool described next.

4.4 Retrieval of solved days and the optimiser tool

Case library. Retrieval gives the agents examples of how similar days were solved best. The case library holds 320 cases: 20 price days for each combination of price pattern and household load of the earlier factorial design, each under its two peak limits, 3.5 and 8 kW. The days are drawn from the hourly prices up to 30 September 2025, excluding every day of every benchmark, so no evaluated day is in the library. Each case is composed like a benchmark scenario of that design (Section 5.1), with the same REFIT cycle pool, price patterns and loads, and solved with the MILP of Section 4.5. The library was not rebuilt for the final benchmark, so it contains no electrified day, and its cases keep the windows of the earlier design rather than the households' usual run times. A deployed system would build such a library from the household's own past days in the same way.

Retrieval. Each day is described by two vectors: the shape of its price curve (24 hourly consumer prices, standardised to mean zero and unit variance) and six structural values (the number of appliances, their combined peak power relative to the headroom under the limit, the mean window length, the share of appliance pairs whose windows overlap, whether the individually cheapest starts would exceed the limit, and the relative price spread). The similarity of two days is the mean of the cosine similarity of the price shapes and e^{-2d}, where d is the Euclidean distance between the structural values. The two most similar cases are retrieved. The description and the weights were set once, before any evaluation run, and were not tuned. The retrieval is deterministic and needs no embedding model.

Worked examples. Each retrieved case is turned into a short text in the agents' own vocabulary: the number of appliances, the peak limit and the price range; whether the individually cheapest starts would exceed the limit; and the optimal start and end of every appliance with its own cheapest start and the extra cost of the difference. The text is appended to the Scheduler's message (and to the single agent's) with a note that the examples show patterns of staggering, not answers, since the day's own prices and loads differ.

Optimiser tool. The Scheduler's prompt asks it to call `optimize_schedule` (Table 4.2) in its first response with all flexible appliances, together with `get_spot_prices`, and to return the solver's start times. The verification step checks every schedule independently

of the tool that produced it, as Jonah et al. (2026) recommend, and returns any problem to the Scheduler, which then has the cost and conflict tools to repair it. This follows the division of work of Li et al. (2023): the solver searches, the agent formulates the call and repairs what the check reports, and the Orchestrator explains the result to the household.

The pipeline reached this form over several design iterations (Section 5.3.1): a four-agent pipeline without retrieval or the optimiser tool, then with retrieval, then with the optimiser tool; then two agents in a deterministic harness; and finally changes that shortened the response time and the prompts without changing what the agents do. The single agent in the harness was built before those last changes, for the comparison of one agent with two, and every change since was applied to both designs alike.

4.5 MILP solver

The MILP solver computes the optimal schedule of a day. It is used in three roles: it confirms during benchmark construction that every scenario has a feasible schedule, it solves the days of the case library, and it is the solver behind the optimiser tool. Its schedules also give the upper bound on the saving in the result tables. The model is the standard time-indexed formulation of appliance scheduling with binary start variables (Sou et al., 2011). The day is divided into slots $t \in \mathcal{T} = \{0, \ldots, 95\}$ of $\Delta = 0.25\,\text{h}$. Each flexible appliance $a \in \mathcal{A}$ has a power profile $p_{a,0}, \ldots, p_{a,d_a-1}$ in kW over its cycle of d_a slots, an earliest start slot e_a and a deadline slot ℓ_a by which the cycle must end. Its admissible start slots are

$$\mathcal{S}_a = \{\, s \in \mathbb{Z} : e_a \le s \le \ell_a - d_a \,\}.$$

The model uses binary start variables $x_{a,s} \in \{0,1\}$, equal to one if appliance a starts in slot $s \in \mathcal{S}_a$. With the consumer price π_t in €/kWh, the base load b_t in kW and the household peak limit $\bar{P}$ in kW, the model is

$$\min_{x} \quad \sum_{a\in\mathcal{A}} \sum_{s\in\mathcal{S}_a} c_{a,s}\, x_{a,s}, \qquad c_{a,s} = \Delta \sum_{k=0}^{d_a-1} p_{a,k}\, \pi_{s+k}, \tag{4.1}$$

$$\text{s.t.} \quad \sum_{s\in\mathcal{S}_a} x_{a,s} = 1 \qquad \forall a \in \mathcal{A}, \tag{4.2}$$

$$b_t + \sum_{a\in\mathcal{A}} \sum_{\substack{s\in\mathcal{S}_a \\ 0\le t-s<d_a}} p_{a,t-s}\, x_{a,s} \le \bar{P} \qquad \forall t \in \mathcal{T}, \tag{4.3}$$

$$x_{a,s} \in \{0,1\} \qquad \forall a \in \mathcal{A},\ s \in \mathcal{S}_a.$$

The objective (4.1) is the consumer cost of the flexible cycles, with $c_{a,s}$ the precomputed cost of starting appliance a in slot s. Constraint (4.2) requires exactly one cycle per appliance, and constraint (4.3) keeps the total household load within the peak limit in every slot. Because a decision fixes the start of a whole cycle, each cycle runs without interruption once started, and restricting the variables to $\mathcal{S}_a$ keeps every cycle within its

user window, so neither property needs a separate constraint. The model is solved with PuLP and the CBC solver in well under a second per scenario. Its correctness was verified against exhaustive enumeration of all start combinations on small scenarios.

4.6 Technology stack

Table 4.3 lists the software used to build and evaluate the artifact, with the versions used for the reported results. All components except the commercial language model are open source. The API cost of the commercial model is an outcome of the evaluation and is reported with the results in Chapter 5.

The evaluation uses GPT-6 Luna, the small model of OpenAI's current generation, which costs USD 0.10 per million input tokens and USD 0.50 per million output tokens; input that repeats the start of an earlier prompt is read from a cache and costs USD 0.01 per million tokens (OpenAI, 2026). The API cost of every run is computed at these rates. OpenAI also charges for writing input to the cache (USD 0.125 or 0.25 per million tokens, depending on how long it is kept), which the recorded cost does not include; billing every uncached input token at the higher write rate would raise the mean cost of a recommendation on the held-out set from USD 0.0004 to at most USD 0.0007. OpenAI publishes no dated snapshot of GPT-6 Luna, so the model name returned with every response is recorded with the run. The model is called without a reasoning phase (reasoning effort *none*): with its default setting the API rejects a temperature parameter, whereas without a reasoning phase it accepts a temperature of zero. The agents need no reasoning phase because the optimiser tool performs the search. The open-weight model gpt-oss-20b, run on local hardware, is used in one exploratory run (Section 5.4).

Table 4.3 Technology stack.

Component	Tool and version	Purpose
Programming language	Python 3.12	implementation of the artifact, baselines and analysis
Agent orchestration	LangGraph 1.2, LangChain 1.4	state graph of the pipeline, ReAct agents, tool calling and schema-validated outputs
Commercial LLM	GPT-6 Luna (reasoning effort none), OpenAI API	model of the evaluation: the two-agent pipeline, the single agent in the harness and the LLM without tools
Open-weight LLM	gpt-oss-20b, Ollama 0.40	open-weight model hosted on local hardware, used in one exploratory run of both agent designs
MILP solver	PuLP 3.3 with CBC	benchmark feasibility check, case library and the optimiser tool
HEMS optimizer	EMHASS 0.18.4, Docker	open-source home energy management optimizer compared in SQ2, called through its REST interface
Data processing	pandas 3.0, NumPy 2.5	REFIT cycle extraction, price series and scenario construction
Tracing	LangSmith	recording the reasoning trace and tool calls of runs during development
Statistics	SciPy 1.18	Wilcoxon signed-rank, Mann–Whitney and TOST equivalence tests, effect sizes and Spearman correlation

5. Evaluation and Results

This chapter presents the evaluation of the artifact, corresponding to DSR Phases 4 and 5. Section 5.1 describes the benchmarks: the appliance and price data, how a scenario is built, and the two factorial benchmarks, an exploratory set and the held-out set Q3. Section 5.2 uses the MILP solver alone to examine Finland's move to fifteen-minute electricity prices, quantifying how much of the achievable saving depends on fifteen-minute rather than hourly price information; this shows where differences between schedulers can be expected before any agent is evaluated. Section 5.3 demonstrates the pipeline on one day (DSR Phase 4) and summarises the design iterations that led to it. Section 5.4 reports the exploratory runs, including the comparison of the pipeline with the single agent in the same harness, and Section 5.5 reports the pre-registered test on the held-out set Q3, which answers SQ1 and SQ2.

5.1 Dataset and scenario design

Each scenario describes one household-day in 96 fifteen-minute slots (00:00–24:00 local time): a price curve, a household base load, a set of flexible appliance cycles with the times at which each is ready and must be finished, and a household peak load limit. Appliance cycles, base loads and usual run times come from measured UK households, and prices from the Finnish day-ahead market. Every benchmark is generated by a deterministic, seeded build procedure, so it can be reproduced exactly from the source data.

5.1.1 Appliance data

Appliance profiles are drawn from the cleaned REFIT Electrical Load Measurements dataset (Murray et al., 2017; Murray & Stankovic, 2016), which records the whole-house aggregate and nine individually monitored appliances at 8-second intervals in 20 UK households between 2013 and 2015. The cleaned release corrects timestamps for daylight saving time and re-aligns appliance monitor columns where a monitor was moved between appliances during the study. The appliance-to-column mapping follows the dataset documentation.

Eighteen of the twenty households are used in the evaluation. The other two do not meet the condition that a household must provide realistic cycles of the appliances to be scheduled. The first has no monitored washing machine, dishwasher or tumble dryer. The second is excluded after inspection: its washing-machine monitor recorded no heating phase in any of its 246 detected cycles (no fifteen-minute mean above 1.2 kW, against 22–70 % of cycles in the other households), so its profiles do not represent the water-heating load that dominates the cost and peak demand of a wash. The washer-dryers in four households are not used as profiles, because they run washing and drying as a

single cycle; their consumption is nevertheless removed from the base load.

Individual appliance cycles are extracted from each monitor by thresholding the one-minute mean power at 20 W and merging pauses shorter than 30 minutes for dishwashers (whose programmes contain a passive drying pause) and 15 minutes for washing machines and tumble dryers (which are often reloaded within minutes). Cycles shorter than 20 minutes, longer than 300 minutes or below 0.1 kWh are discarded, and each remaining cycle is converted into a fifteen-minute mean power profile. This yields 6,172 washing-machine cycles from 19 monitored machines, 4,702 dishwasher cycles from 15 machines and 1,940 tumble-dryer cycles from 7 machines. To prevent heavily used appliances from dominating the benchmark, a scenario first draws a monitored appliance of the required type and then one of its cycles. The household base load is the median, per fifteen-minute slot of the day, of the aggregate minus all shiftable appliance monitors, computed after discarding readings that the dataset flags as inconsistent (sub-metered sum exceeding the aggregate). The start time of every detected cycle is also kept: it shows when each household usually runs each appliance, which sets the appliance windows (Section 5.1.3).

5.1.2 Electricity price data

Day-ahead spot prices for the Nord Pool FI bidding zone come from two public services that republish Nord Pool data. The hourly prices of the exploratory benchmark and the case library, 1 January 2022 to 30 September 2025, come from Sähkötin (Pakastin Oy, 2026); each hourly value is repeated across the four fifteen-minute slots of its hour. The fifteen-minute prices of the held-out benchmark, from 1 October 2025, come from Elering (Elering AS, 2026). All values are in EUR/MWh excluding VAT, and days with a daylight-saving transition are dropped. Both sources were checked against the official day-ahead prices of the ENTSO-E Transparency Platform for the whole period (ENTSO-E, 2026). The Elering prices are identical to the official ones at all 67,411 timestamps. The Sähkötin prices are identical for every hour before October 2025 except one hour on a daylight-saving transition day, which no benchmark uses. Every price in the scenarios of the exploratory and the held-out benchmarks therefore matches the official data.

Scheduling cost is computed with the consumer price rather than the raw spot price. For spot price s, the consumer price per kWh is

$$p = (s + m + d + \tau)(1 + v),$$

with a retail margin $m = 0.5$ c/kWh, a distribution fee $d = 5.0$ c/kWh, the electricity tax $\tau = 2.253$ c/kWh and VAT $v = 25.5\,\%$, the general rate since 1 September 2024 (Finnish Tax Administration, 2026). These are representative values for a Finnish household on a spot-price contract. The tax is that of tax class I including the security-of-supply fee, valid until 31 March 2026; on 1 April 2026 the fee rose from 0.013 to 0.085 c/kWh, raising the tax to 2.325 c/kWh (Caruna, 2026). The benchmark keeps one tariff for all days, including a single VAT rate, so that scenarios from different periods remain comparable. Because

every component is constant per kilowatt-hour, the choice of values does not change the optimal schedule, the ranking of the systems or the savings captured, only the size of the optimality gap (Section 3.3). Using the consumer price keeps costs positive on days with negative spot prices, which keeps the optimality gap defined in Section 3.3 meaningful. Only the energy of the flexible appliances is costed, since the base load costs the same under every schedule.

5.1.3 Scenario construction

Household loads. A scenario has one of three household loads. The *light* load is a dishwasher and a washing machine. The *heavy* load is two washing cycles, two drying cycles and a dishwasher. The *electrified* load is the heavy load of the same day with two loads added that make Finnish houses reach 8 kW: an electric car charge and a sauna. REFIT provides three flexible appliance types, so a heavy scenario contains two cycles of the same type, such as two washing loads.

Windows from usual run times. REFIT records power, not when a household would allow an appliance to run, so the window of each appliance is taken from the household's own habit: the clock times at which its REFIT cycles of that appliance started. The appliance is ready at the 25th percentile of these start times and must be finished by the 75th percentile plus the length of the cycle, and the window always leaves at least two hours beyond the cycle and ends by midnight. A household that owns no appliance of the required type uses the median of these percentiles over all households that do. The habits are kept as local clock times: a household that usually starts its washing machine at 10:00 keeps that time on the Finnish price day. Without scheduling, every appliance starts when it is ready. Windows built this way are 7.9 to 9.2 hours long on average.

Car and sauna. The car charge is flexible: a constant 3.7 kW for two hours (7.4 kWh), the power of a single-phase home charge point on a 16 A circuit. The energy covers about 37 km at 0.2 kWh/km, close to the 34 to 40 km that Finns travel per person and day (Traficom, n.d.). The car is ready on arrival at 17:00 and must be charged by midnight, because the benchmark day ends then; Motiva recommends charging overnight (Motiva, 2026). Without scheduling, it charges on arrival and finishes at 19:00, as the sauna starts. A three-phase 11 kW charger was not used, because it alone exceeds the 8 kW limit and could not be scheduled under a hard limit. The sauna is fixed: the household decides when to bathe, so a 6.8 kW heater, a common size for saunas of 5 to 10 m^3 (K-Rauta, 2026), runs from 19:00 to 20:30, one hour of heating and thirty minutes of bathing (ELY Centre, n.d.). Its power is added to the base load: it counts towards the peak limit but not towards the cost, like every load that cannot be moved, and the household's request states it. The arrival and sauna times are assumptions, the same on every day, since no measured Finnish data on them were available.

Peak limit. The household peak load limit, including the base load, applies in every fifteen-minute slot. No Finnish rule caps the power a household may draw below the capacity of its connection, and that capacity does not bind: the smallest standard main fuse, 3×25 A (Elenia, 2026), allows about 17 kW, far above these appliances. The only limit with a regulatory source is a price threshold. The Energy Authority's regulation on distribution fees, published on 2 February 2026, makes every network company harmonise its pricing by the start of 2029 at the latest: where a distribution product for households has a power charge, the charge is based on the highest 60-minute average power of the month above a threshold of 8 kW (Energy Authority, 2026). The power charge remains optional, since a product without one must still be offered (Helen Sähköverkko, 2026b). Until then the network companies set their own products. Helen Sähköverkko's time-based distribution product, for example, charges €1.57 per kW and month, excluding VAT, for the third-highest hourly average power of the month, with 80 % of night-time power counted (Helen Sähköverkko, 2026a). Oulun Energia Sähköverkko, in contrast, charges for power only in its tariffs for larger low-, medium- and high-voltage connections; its general, time-of-use and seasonal products for households have no power charge (Oulun Energia Sähköverkko, 2026).

The benchmarks therefore take their peak limit from the regulation, 8 kW, and treat it as a hard limit in every fifteen-minute slot. This is stricter than the regulation in time resolution (15 instead of 60 minutes) and in kind (a hard limit instead of a price); replacing it with a priced peak is left for future work. A scenario's limit would be raised if its optimal schedule were infeasible, but this was never necessary. During development a stricter limit of 3.5 kW was also used, a stylized cap without a regulatory or technical source; it was dropped for that reason. Under 8 kW the light and heavy REFIT loads rarely compete for capacity, which is why the electrified load was added.

The build procedure verifies with the MILP solver that every scenario has a feasible schedule. A scenario *requires coordination* when running every appliance at its individually cheapest start time would exceed the peak limit; these scenarios form the basis of the conflict resolution rate. Table 5.1 reports the characteristics of both benchmarks per household load.

Table 5.1 Benchmark characteristics per household load (24 scenarios each). Cost without scheduling: median per scenario; saving: of the optimal schedule, pooled.

Benchmark	Load	Appliances	Require coordination	Cost without scheduling	Optimal saving
Exploratory	Light	2	0/24	€0.33	21.8 %
	Heavy	5	2/24	€1.19	17.5 %
	Electrified	6	8/24	€2.62	23.7 %
Held-out Q3	Light	2	0/24	€0.27	15.7 %
	Heavy	5	2/24	€0.90	15.7 %
	Electrified	6	9/24	€1.85	16.0 %

Starting every appliance when it is ready stays within the limit in all 144 scenarios, so the reference cost is always that of a valid plan. Coordination matters mainly on electrified days: there, the car and the dryers compete with the sauna for the capacity left in the evening.

5.1.4 Factorial benchmarks

The benchmarks follow a factorial design in which the price pattern and the household load vary separately, so that an effect can be attributed to one of them. The *price pattern* has four levels: stable (the lowest quarter of days by within-day standard deviation), volatile (the top 15 %), spike (the days with the highest daily maximum) and negative (at least one hour below zero). Six price days are drawn for each combination of price pattern and the light or heavy load, so a benchmark has 48 days; each day is assigned a REFIT household in turn. The electrified load reuses the heavy-load days, so that its effect is measured on the same days, prices and laundry cycles. A benchmark therefore has 72 scenarios.

Two benchmarks were built with this design, and no day appears in both. The exploratory benchmark uses the hourly prices up to 30 September 2025, and its days were used throughout the design iterations. The held-out set Q3 uses the native fifteen-minute prices from 2 October 2025 to 24 September 2026. It was built on 21 October 2026, and only the characteristics in Table 5.1 were computed before the pre-registration was committed; no system was scored on it before. None of its days is in the exploratory benchmark or in the case library for retrieval (Section 4.4). Q3 is therefore held out in time as well: all its days come after every day on which the design was developed, so the test imitates using the system on future days. In Q3, the cost of starting every appliance when it is ready has a median of € 0.90 per scenario (interquartile range € 0.37 to € 1.66, maximum € 4.49).

5.2 Value of 15-minute price information

Since 1 October 2025 the Nord Pool day-ahead market settles Finnish electricity in fifteen-minute periods. The exploratory benchmark and the case library use hourly prices, since they cover the years before. The held-out set Q3 has native fifteen-minute prices, and in them prices genuinely vary within the hour.

For each Q3 scenario, two optimal schedules were computed with the MILP solver. The first is planned on the real fifteen-minute prices. The second is planned on hourly averages, which is all the price information a household had before the switch, and is then settled at the real fifteen-minute prices. Hourly prices make the four quarters of an hour tie, so ties are broken towards the earliest start. The cost difference between the two schedules is the saving that is only reachable with fifteen-minute price information. Table 5.2 reports the savings of both schedules relative to starting every appliance when it is ready, the additional cost reduction from fifteen-minute information, and the share of the achievable saving that requires it, by price pattern and by household load; the rows weight scenarios by their cost.

Table 5.2 Optimal schedule cost with hourly vs. 15-minute price information (MILP, 72 scenarios).

Condition	Intra-hour spread (c/kWh)	Saving, hourly plan	Saving, 15-min plan	Additional cost reduction	Share of saving needing 15-min	Scenarios improved
Stable prices	0.50	3.0 %	3.4 %	0.4 %	10.6 %	16/18
Volatile prices	2.71	22.8 %	23.6 %	1.0 %	3.4 %	14/18
Price spike	3.03	19.3 %	20.1 %	1.0 %	3.9 %	18/18
Negative prices	0.30	1.1 %	1.2 %	0.0 %	4.0 %	17/18
Light load	1.90	13.6 %	15.7 %	2.4 %	13.2 %	20/24
Heavy load	1.50	15.0 %	15.7 %	0.9 %	4.6 %	23/24
Electrified load	1.50	15.7 %	16.0 %	0.5 %	2.5 %	22/24
All	1.63	15.3 %	15.9 %	0.7 %	3.9 %	65/72

Fifteen-minute price information lowers the optimal cost in 65 of the 72 scenarios, but the effect is modest: 0.7 % of the cost overall, or 3.9 % of the achievable saving. The remaining 96 % of the saving is available to a scheduler that plans on hourly prices, since most of the value lies in moving appliance cycles between hours rather than between quarters of an hour. The additional reduction grows with the intra-hour price spread (Spearman $\rho = 0.65$ across scenarios) and is largest on volatile and spike days with the light load, where it reaches 7.8 % in the single largest case (Q3-VL02-80, 9 April 2026). Its share of the achievable saving is largest for the light load (13.2 %) and on stable days (10.6 %), where hourly price differences are small, so intra-hour variation forms a larger part of the opportunity. In absolute terms the gain is under one cent per household and day on average, and at most nine cents. The schedules nevertheless differ: 190 of the 312 appliance start times change between the two plans. Differences between the schedulers on Q3 can therefore come from fifteen-minute prices, but mostly from how they place cycles between hours and under the peak limit.

5.3 Demonstration results

Table 5.3 shows what each system recommends for one day of the held-out set Q3, scenario Q3-PE01-80 (15 October 2025). It is a spike day with the electrified load. Two washing cycles and a dishwasher are ready from about 08:00, two drying cycles from 11:15, and the car from 17:00; the sauna heats from 19:00 to 20:30, and the household's total load, including a base load of 0.2 to 0.8 kW and the sauna's 6.8 kW, must stay within 8 kW. The consumer price peaks at 40 c/kWh between 09:00 and 10:00, just after the appliances are ready, stays above 21 c/kWh until 19:00 and falls to 10 c/kWh by midnight. Starting every cycle when it is ready would cost €3.69; it stays within the limit, because the car finishes charging as the sauna starts.

Table 5.3 Daily recommendations for scenario Q3-PE01-80 (15 October 2025, spike price pattern, electrified load, limit 8 kW; run 1 of the systems with a language model, GPT-6 Luna). Saving in percent of the cost without scheduling, €3.69; an invalid schedule saves nothing.

System	Washing 1	Washing 2	Drying 1	Drying 2	Dishwasher	Car	Peak (kW)	Saving
No scheduling	08:15	08:15	11:15	11:15	08:00	17:00	7.31	–
Multi-Agent Pipeline	13:30	13:30	17:00	17:00	17:15	22:00	7.31	41.2 %
Single agent in the harness	13:30	13:30	17:00	17:00	17:15	22:00	7.31	41.2 %
EMHASS, measured profiles	13:30	13:30	17:00	17:00	17:15	22:00	7.31	41.2 %
Smart timer (own heuristic)	13:30	13:30	17:00	17:00	17:15	22:00	7.31	41.2 %
EMHASS, nominal power	13:30	13:30	17:00	17:00	17:00	22:00	7.31	40.5 %
Cheapest window	14:00	13:00	16:00	16:00	18:00	22:00	8.39	invalid
Cheapest-hours automation	08:15	11:45	12:45	12:45	13:30	19:45	11.00	invalid
LLM without tools	01:00	05:00	11:15	12:45	13:15	22:00	7.31	invalid
Optimum (upper bound)	13:30	13:30	17:00	17:00	17:15	22:00	7.31	41.2 %

The Multi-Agent Pipeline returned the optimal schedule. It moves the washing past the morning price peak to 13:30, runs the dryers and the dishwasher from 17:00, before the sauna, and charges the car at 22:00, when the price is lowest and the sauna has ended; it saves €1.52, or 41.2 % of the cost without scheduling. The single agent in the harness, EMHASS with measured profiles and the smart timer found the same schedule. EMHASS configured with a constant nominal power started the dishwasher a quarter of an hour earlier and saved 40.5 %. The cheapest-hours tools found cheap hours for each appliance but ignored the sauna. The cheapest window starts the dishwasher at 18:00, so its cycle runs into the sauna's heating at 19:00 and the load reaches 8.4 kW; the cheapest-hours automation charges the car from 19:45, during the sauna, and the load reaches 11.0 kW. Both schedules break the limit and therefore count as saving nothing. The LLM without tools moved the car to 22:00 and stayed within the limit, but it started both washing cycles at night, hours before the household had them ready, which the verification of a schedule would have caught.

In a separate demonstration run on the same day with the household summary switched on, the pipeline returned the same schedule in 11.8 seconds, and the Orchestrator wrote: "The washing machines run at 13:30, both tumble dryers at 17:00, the dishwasher at 17:15, and the EV charger at 22:00. The scheduled cycles cost €2.172, saving €1.520 (41 %) compared with starting each appliance as soon as it's ready. The peak-load check confirms there are no conflicts: the maximum load is 7.31 kW, below the 8.00 kW limit." The summary is written from the cost figures and the result of the final peak load check (Section 4.2), so its times, costs and statement about the limit are correct. In the evaluated run, without the summary, the pipeline took 8.4 seconds, 4,276 tokens and $0.0005 in API fees with four tool calls, and its schedule passed the verification step without a feedback round; the single agent took 4.4 seconds, 4,175 tokens and $0.0004 with three.

5.3.1 Design iterations

The final pipeline is the result of fourteen build-and-evaluate cycles, in line with the iterative design and demonstration activities of DSR (Peffers et al., 2007). Table 5.4 lists them. The first five were developed on a first benchmark of 48 scenarios in six types, iterations 6 to 11 on factorial exploratory benchmarks, and the last three were checked on a few exploratory scenarios and then run on the exploratory benchmark of Section 5.1.4. Because the design was refined on these days, they are treated as exploratory, and the hypotheses were tested only on the held-out set Q3. Every change is a general mechanism that applies to all scenarios and, where it concerns errors of the language model, equally to every agent; none was tuned to an individual scenario.

The single agent in the harness was built after iteration 12, when the deterministic parts of the pipeline were moved into a shared harness module, and iterations 13 and 14 were applied to both designs alike. Before the final runs, one defect was also found and fixed: inside the pipeline's graph, the recovery nudge was sent without the agent's conversation, so a model that needed it could not answer. It now carries the whole conversation, and every run reported in this chapter was made after the fix. It matters most for the local open-weight model of Section 5.4, which needs a nudge in almost every run.

Three observations follow from the iterations. First, most problems concerned reliability rather than cost: the agents usually found cheap start times, but a schedule had to be complete, within the peak limit and within every window before its cost counted. This is why the verification step grew from a peak-load check into a check of all three conditions, and why it is deterministic. Second, once the optimiser tool solved the whole day, work that the agents had done with a language model turned out to be a fixed procedure, and moving it into the harness removed two agents without changing what the pipeline decides. Third, the last iterations show that the response time and the API cost are set by the number and length of the model calls, not by the tools, so they were reduced by sending less and calling less, not by changing the model or the optimiser.

Table 5.4 Design iterations of the pipeline.

	Problem observed	Change
1	The Conflict Resolver moved appliances back and forth until it ran out of steps, and the summary could report success for a schedule that still exceeded the peak limit.	Automatic mode of `adjust_schedule`; the summary is written from the result of a final peak-load check.
2	Models ended their turn in prose without the required structured output, or the provider rejected a malformed tool call.	Recovery nudge, at most two per agent and run, applied identically to every agent and model and logged.
3	The Orchestrator's brief sometimes omitted an appliance, which was then never scheduled.	The daily plan travels with the brief; the verification step checks that every appliance has a start time and returns missing ones.
4	With the step budget divided equally, the Scheduler ran out of steps.	Budget divided by workload.
5	Some schedules started a cycle outside its window.	The verification step also checks every start against its window.
6	Valid schedules were sometimes far from the cheapest.	A cost check that returned such schedules once; it helped on some days and hurt on others and was dropped.
7	Single moves of one appliance could settle on a staggered schedule far from the optimum.	Retrieval of two similar solved days as worked examples, and the optimiser tool.
8	After a feedback round, a schedule could still leave out an appliance.	The verification step places a still missing appliance at its cheapest start within the limit.
9	The response time was almost all waiting for sequential model calls; the tools took under half a second.	Fewer calls: the Scheduler starts with the optimiser and does not re-cost its answer, and the Resolver runs only when a check fails.
10	The Profiler's work was a fixed reading of the household data, and the Resolver was almost never needed.	Two agents: profiling by a deterministic step, the Resolver's tools given to the Scheduler for the feedback round.
11	With native tool calling, the agents wrote no Thought before their tool calls, so the trace held only Actions and Observations.	An optional explicit Thought argument in every tool; reported as a variant, not part of the final design.
12	The trace did not show the prices the schedule was based on.	The Scheduler reads the day's prices once; repeated reads come from a cache.
13	Each model call carried the full price table, and the first two tool calls took two round trips.	Prices and optimiser called in one response; a short day summary instead of 96 prices; shorter outputs; the household summary left out of the evaluated runs.
14	Every model call carried the descriptions of all tools and long profiles and examples.	Only the two tools of a normal run in the first round, the others with feedback; one line per appliance; shorter examples; start times only in the answer.

5.4 Exploratory results

Before the held-out run, every system was run on the 72 scenarios of the exploratory benchmark with the final design, the systems with a language model three times per scenario with GPT-6 Luna. The light and heavy scenarios were run first; the electrified ones were added once that load had been designed (Section 5.1.3), with unchanged code. The results are exploratory, since the design was developed on these days.

The picture is the same as on the held-out set (Section 5.5). Both agent designs were valid in all 216 runs and saved 21.9 % of the cost without scheduling, exactly the saving of the optimal schedule, as did EMHASS with measured profiles (within 0.01 percentage points) and the smart timer. The cheapest window saved 20.2 % and was valid on 90 % of the days, the cheapest-hours automation 6.8 % and 86 %, and the LLM without tools 6.2 % and 82 % of its runs. With the electrified load, the cheapest window was valid on 75 % of the days and the cheapest-hours automation on 63 %, and the latter saved only 3.0 % against 23.7 % for the optimum. Applied to these data, the registered tests of Table 3.2 supported eight of the nine hypotheses. H2d, a larger advantage over the cheapest-hours automation on electrified days, was supported here (rank-biserial 0.34, adjusted $p = 0.04$) but not on the held-out days, and H2e was not supported in either set.

One agent or two. The single agent in the harness returned the same schedule as the pipeline in every run. It took 4.9 seconds on average against 7.9 for the pipeline (median 4.7 against 7.3), cost \$0.00024 per run against \$0.00040, and used about 100 fewer tokens per run (3,731 against 3,828); all three differences are significant (Wilcoxon, $p < 0.001$). Neither design needed a feedback round or a recovery nudge in any run.

An open-weight model run locally. Both agent designs were also run with gpt-oss-20b, a 20-billion-parameter open-weight model, on a laptop with a consumer graphics card, on the 48 light and heavy scenarios, three runs each. Both designs were valid in all 144 runs and reached the optimal schedule in every one, so the result does not depend on a commercial model. The local model was slower: the median response time was 34.3 seconds for the pipeline and 22.0 seconds for the single agent, with 90 % of the runs within 59 and 33 seconds. It also needed the recovery nudge in every run, because it answered once in prose before giving the schedule in the required form, and in two runs on one day the pipeline's Scheduler returned no usable answer even after the nudges, so the verification step sent the problem back and the feedback round produced a valid schedule. The harness thus absorbed the local model's errors without loss of cost or validity. The model was not run on the electrified days or on the held-out set, and the LLM without tools was not run with it.

5.5 Confirmatory evaluation on the held-out set Q3

The hypotheses of Table 3.2 were fixed in a pre-registration that was committed to the project repository on 21 October 2026 together with the held-out set, the code, prompts, tools, case library and analysis script. The run on Q3 followed the same day, with nothing changed in between. The Multi-Agent Pipeline, the single agent in the harness and the LLM without tools used GPT-6 Luna with three runs per scenario, EMHASS and the rule-based systems one run each, and the analysis was run exactly as registered. The run cost USD 0.16 in API fees for all 648 language model runs together.

Registered tests. Seven of the nine hypotheses were supported (Table 5.5). The pipeline saved more than the LLM without tools and was valid more often (H1a, H1b), its mean response time and API cost per recommendation were far below the limits (H1c), it saved more than both cheapest-hours tools (H2a, H2b), and it saved the same as EMHASS within the margin (H2c). The two hypotheses about the household load were not supported: the pipeline's advantage over the cheapest-hours automation and over the cheapest window was not significantly larger on electrified days than on the others (H2d, H2e).

Table 5.5 Pre-registered confirmatory tests on the held-out set Q3 (GPT-6 Luna, $N = 72$ scenarios; estimate = mean per-scenario difference, first minus second system, or the mean for H1c; p Holm-adjusted within SQ1 and within SQ2).

	Hypothesis	Estimate	Effect	p	Supported
H1a	Pipeline saves more than the LLM without tools	7.40 pt	$r = 0.87$	< 0.001	yes
H1b	Pipeline is valid more often than the LLM without tools	23.1 pt	$r = 0.59$	< 0.001	yes
H1c	Mean response time below 60 s	7.4 s		< 0.001	yes
H1c	Mean API cost below \$0.10	\$0.0004		< 0.001	yes
H2a	Pipeline saves more than the cheapest window	4.65 pt	$r = 0.87$	< 0.001	yes
H2b	Pipeline saves more than the cheapest-hours automation	5.99 pt	$r = 0.87$	< 0.001	yes
H2c	Pipeline saves the same as EMHASS within ± 1 pt	0.002 pt		< 0.001	yes
H2d	Advantage over cheapest hours larger on electrified days	6.66 pt	$r_{rb} = 0.21$	0.30	no
H2e	Advantage over cheapest window larger on electrified days	2.40 pt	$r_{rb} = -0.10$	0.51	no

All systems. Table 5.6 reports every system on the 72 held-out scenarios. Both agent designs produced a valid schedule in all 216 of their runs and saved 15.92 % of the cost without scheduling, which is the saving of the optimal schedule: their schedules cost exactly the optimum in every run, without a single feedback round or recovery nudge.

EMHASS with measured profiles and the smart timer also reached the optimum, the former within 0.01 percentage points: it differed from the pipeline on 9 of the 72 days, by at most 0.07 points. The equivalence with EMHASS also holds at margins of 0.5 and 2 points and in the Wilcoxon version of the test ($p < 0.001$ in every case). Measured as the gap to the optimum, the cheapest window cost 8.7 %, the cheapest-hours automation 13.5 % and the LLM without tools 14.1 % more than the optimal schedule, invalid schedules counted at the cost without scheduling. The cheapest-hours tools were valid on 83 to 86 % of the days and captured only 29 to 54 % of the achievable saving, and the LLM without tools was valid in 77 % of its runs and captured 26 %. Over the 72 days, following the pipeline would have saved € 14.48, about € 0.20 per household and day, for API fees of about $0.0004 per recommendation.

Table 5.6 All systems on the held-out set Q3 (72 scenarios; systems with a language model: GPT-6 Luna, three runs each). Saving pooled over runs; share = share of the achievable saving captured; coordination = conflict resolution rate, the valid share on the 11 scenarios that require coordination.

System	Valid	Saving	Share	Coordination	Mean time (s)	API cost per run
Multi-Agent Pipeline	100 %	15.92 %	100 %	100 %	7.4	$0.00040
Single agent in the harness	100 %	15.92 %	100 %	100 %	5.0	$0.00025
EMHASS, measured profiles	100 %	15.91 %	100 %	100 %	–	–
EMHASS, nominal power	100 %	15.36 %	96.5 %	100 %	–	–
Smart timer (own heuristic)	100 %	15.92 %	100 %	100 %	–	–
Cheapest window	86.1 %	8.63 %	54.2 %	27.3 %	–	–
Cheapest-hours automation	83.3 %	4.57 %	28.7 %	54.5 %	–	–
LLM without tools	76.9 %	4.07 %	25.6 %	69.7 %	2.2	$0.00011
Optimum (upper bound)	100 %	15.92 %	100 %	100 %	–	–

By household load and price pattern. Table 5.7 breaks the saving down by household load. The rules fall behind as the load grows: the cheapest-hours automation saved 11.1 % with the light load, 9.5 % with the heavy load and 1.6 % with the electrified load, where only 54 % of its schedules were valid, and the cheapest window saved 12.4, 9.2 and 7.9 %, valid on 71 % of the electrified days. Their schedules break the limit when an appliance runs into the sauna's heating or the car charges during it, as in the demonstration. They also lose on light-load days, where every schedule is valid: neither rule chooses a start by the cost of the whole cycle at fifteen-minute prices. The cheapest window looks for the cheapest block of whole hours, and the cheapest-hours rule switches a device on whenever the price falls below a threshold, which can start a cycle that then runs on into dearer slots. This is why H2d and H2e were not supported: the advantage over the rules is not concentrated on electrified days but present on every load, and its size varies more from day to day than between the loads. Over cheapest hours the mean advantage was 6.7 percentage points larger on electrified days, but the rank test does not separate the groups (rank-biserial 0.21, $p = 0.30$); over the cheapest window the mean advantage was 2.4 points larger, but in ranks the electrified days lie slightly below the others (-0.10). The price pattern sets the size of the stakes: the optimum saved 23.6 % on volatile days, 20.1 %

on spike days, 3.4 % on stable days and 1.2 % on negative-price days, and the pipeline reached it under every pattern.

Table 5.7 Saving by household load on the held-out set Q3 (pooled; in parentheses the share of valid runs where below 100 %).

System	Light	Heavy	Electrified
Multi-Agent Pipeline	15.7 %	15.7 %	16.0 %
Single agent in the harness	15.7 %	15.7 %	16.0 %
EMHASS, measured profiles	15.7 %	15.7 %	16.0 %
Smart timer (own heuristic)	15.7 %	15.7 %	16.0 %
Cheapest window	12.4 %	9.2 % (88 %)	7.9 % (71 %)
Cheapest-hours automation	11.1 %	9.5 % (96 %)	1.6 % (54 %)
LLM without tools	8.2 % (82 %)	5.4 % (79 %)	2.9 % (69 %)
Optimum (upper bound)	15.7 %	15.7 %	16.0 %

One agent or two. The single agent in the harness returned the same schedules as the pipeline in every run, with the same saving and validity, but it was faster and cheaper. Its mean response time was 5.0 seconds against 7.4 (median 4.9 against 7.2), its API cost $0.00025 against $0.00040 per run, and it used about 100 fewer tokens per run (3,736 against 3,834); all three differences are significant (Wilcoxon, $p < 0.001$). The pipeline's extra time and cost come from its additional model call: the Orchestrator's brief, which on these plans only restates the appliance list that the harness already provides. Neither design needed the feedback round or a recovery nudge on the held-out days, so the division into two agents had no work to do that the harness did not already do.

6. Discussion

This chapter interprets the results of Chapter 5. It first answers the main research question and then the two sub-questions that supply its evidence. It then relates the findings to the prior work reviewed in Chapter 2 and draws implications for households and for the design of agent-based home energy management. It ends with the limitations of the study and directions for future research.

6.1 Interpretation of the research questions

6.1.1 RQ: design of the multi-agent system

The main research question asked how a hierarchical ReAct multi-agent system can be designed to give households reliable, cost-saving daily recommendations. The design iterations answer it as much as the final results do, whose evidence is discussed for SQ1 and SQ2 below. The first designs left the search for a good schedule to four agents with tools that move one appliance at a time; they produced valid schedules but not always cheap ones on the days that required coordination. Retrieval of similar solved days helped only partly. The optimiser tool closed the gap, and once it did, two of the four agents turned out to do fixed work: the profiling became a deterministic step and the conflict repairs a feedback round of the Scheduler. The final pipeline of two agents in a deterministic harness then reached the optimum in every held-out run.

The comparison with the single agent in the same harness answers the hierarchical part of the RQ. One agent with the same tools, context and checks returned the same schedules in every run, a third faster and at about 60 % of the API cost. On these problems the hierarchy added a model call but no decision: the Orchestrator's brief restates the appliance plan that the harness already passes on, and its summary can be written by the single agent in the same conversation. The reliability came from what the agents were given to work with, the solver and the checks, not from dividing the work among agents.

Six design principles follow. First, every number the agents reason with should come from a deterministic tool, and the combinatorial search should be handed to a solver; the language model is good at deciding what to compute and at explaining the result, not at searching a space of start-time combinations. Second, every schedule should be checked by a verification step that does not rely on the agents, and returned for revision when it fails. Third, rare but predictable failures of the model, such as a missing appliance, are better closed by a deterministic fallback than by more prompting. Fourth, work that follows a fixed procedure, such as profiling the appliances or retrieving examples, belongs in a deterministic harness around the agents rather than in an agent. Fifth, a model call should be made only where the model has something to decide, and each call should carry only what it needs, because the response time and the cost grow with every call made one after

the other and with every token sent. Sixth, a second agent should be added only when it has decisions of its own: under the first five principles, a single agent in the harness was as reliable as two and faster and cheaper. The hierarchy may still pay off where the agents' work is less fixed, for example without an optimiser tool or with requests stated in ordinary language, which this study did not test.

6.1.2 SQ1: value and cost of the recommendations

SQ1 asked how much the pipeline's recommendations save, how often they are valid, and what they cost to produce, compared with a language model that answers without tools. On the held-out days the pipeline saved 15.9 % of the cost of starting every appliance when ready, the saving of the optimal schedule, in every one of its runs. The same model answering in a single prompt saved 4.1 %, and on average per scenario 7.4 percentage points less (H1a). The difference does not come from the language model, which is the same in both systems, but from how it is used. Without tools the model has to read the prices and base loads of 96 slots, add up power profiles and compare start times in its head; it rarely finds a cheap combination and sometimes ignores a window or the limit. With tools those calculations are exact, and the model only decides what to ask for. This supports the premise of the ReAct approach, that reasoning grounded in tool observations does better than unaided generation (Yao et al., 2023), on a task where the observations are numbers.

Validity separated the two systems as well (H1b). The pipeline was valid in all 216 runs, the LLM without tools in 77 %. In 49 of its 50 invalid runs it started a cycle before the appliance was ready or too late to finish by the deadline; only one broke the peak limit. On electrified days it was valid in only 69 % of its runs. The pipeline's validity does not depend on the model getting every detail right, because its verification step checks every schedule before it is returned; on the held-out days, however, it never had to send one back.

A recommendation cost about $0.0004 in API fees, two hundred and fifty times below the objective of $0.10, and took 7.4 seconds on average and 18 seconds at most, far below the objective of 60 seconds (H1c). Against an average saving of about €0.20 per household and day, the API cost is about 0.2 % of the saving. Both were reached by design rather than by choosing a faster model: the design iterations reduced the number of model calls and the length of each prompt (Section 5.3.1), since almost all of a run's time is spent waiting for the model.

6.1.3 SQ2: conditions under which the system pays off

SQ2 asked under which price patterns and household loads the recommendations save at least as much as existing home energy management tools. Against the cheapest-hours tools the answer is: under every condition. The pipeline saved 4.6 and 6.0 percentage points more per scenario than the cheapest window and the cheapest-hours automation

(H2a, H2b), and more under every price pattern and every load. The comparison changes with the load in kind more than in size. With the light load the rules are always valid and lose only by choosing worse start times; with the electrified load they also break the limit, because they plan each appliance on its own and ignore the sauna and the car, and the cheapest-hours automation saved almost nothing there. Nevertheless, the advantage over the rules was not significantly larger on electrified days (H2d, H2e): it was present on every load and varied more from day to day than between loads. The registered answer to the conditions part of SQ2 is therefore that the pipeline's advantage over the cheapest-hours tools does not depend on the household load in a way this test could detect, while the price pattern sets how much there is to gain, from about 1 % on negative-price days to 24 % on volatile days.

Against EMHASS, the pipeline saved the same under every condition, with a mean difference of 0.002 percentage points (H2c). This is expected, because both solve the same optimization problem in the end, EMHASS with its own solver and the pipeline through its optimiser tool. The difference between them lies outside the cost. EMHASS has to be configured with the power, run time and window of each appliance in numeric form, and it returns a power plan per time step without explanation. The pipeline works from the day's appliance plan, checks its schedule against the limit and the windows, and returns start times with a short explanation for the household. Whether households value that explanation was not measured in this study.

A rule written for this study, the smart timer, placed the appliances one at a time and reached the optimum on every held-out day without a language model. On problems that are fully specified, with an explicit window for every appliance and room under the limit, a well-chosen greedy rule is thus as good as an optimizer. This qualifies the result for SQ2: the advantage of the pipeline over the cheapest-hours tools is an advantage of planning the appliances together under the limit, which a simple peak-aware rule also has, and not of the language model as such.

6.2 Relation to prior work

The results address the open problems that Jonah et al. (2026) listed for LLM-based home energy management. Multi-appliance coordination, which failed for two of the three models of El Makroum, Zwickl-Bernhard, and Kranzl (2026), did not fail once in the final design, but because the coordination was delegated to a solver, not because a hierarchy of agents coordinated better. In their hierarchical system the agents coordinate the appliances themselves; in this study, a hierarchy of two agents and a single agent did equally well once a solver and an independent check were in place. Jonah et al. recommend that an independent deterministic validator check every schedule before it is executed; this thesis implements that recommendation as the verification step, together with a deterministic fallback for a missing appliance. Their agents captured 96.7 to 98.0 % of the savings available between an off-peak timer and an optimum; the final pipeline here captured all of the saving available between no scheduling and the optimum, on a held-out benchmark that varies the price pattern and the household load rather than a single household over a

few weeks.

The final design follows the division of work of OptiGuide (Li et al., 2023), in which a language model uses an optimization solver instead of replacing it. The thesis adds evidence that this division also holds for a household-scale problem and that it improves cost, tokens and response time at once compared with agents that search themselves. Retrieval-augmented generation (Lewis et al., 2020) helped less in the design iterations: solved examples improved the agents' schedules on moderately difficult days but did not enable them to search hard ones, which suggests that examples teach the form of a good solution more easily than the search for it. For multi-agent systems in general (Guo et al., 2024; Hong et al., 2024), the results caution against dividing a task among agents by default: when the arithmetic, the search and the checks are deterministic, a single agent in a harness did the remaining work as well as two, at lower cost.

6.3 Implications for practice

For households, the results suggest caution with the cheapest-hours automations that are widely shared for Home Assistant. They work when the household runs one or two appliances and has power to spare, but they ignore the household's total load: in a home with an electric car and a sauna they broke the 8 kW threshold on 29 to 46 % of the held-out days. Under the power-based distribution tariffs that Finnish network companies are moving towards, which charge for the household's highest hourly power above that threshold, such tools would raise the power charge exactly on those days. A peak-aware method, whether an optimizer such as EMHASS, a peak-aware rule or an agent with an optimiser tool, avoids that.

For developers of home energy management systems, the results argue for a hybrid design rather than for replacing optimization with language models, and for a lean one. The solver guarantees the best schedule; the language model adds the interface: it can take the day's plan, check and explain the result, and could in future take wishes stated in ordinary language. Deterministic work should be done by code around the model, and a single agent is enough for the task studied here. The cost of the language model is negligible compared with the saving, and a response within seconds is fast enough for a day-ahead recommendation, which is computed once after the prices are published.

6.4 Limitations

The study has eight main limitations. First, the evaluation is a simulation on historical data. Real consumption profiles and real Nord Pool FI prices make it more realistic than synthetic scenarios, but the system was not tested in a live deployment, where appliance states change during the day and users may not keep to the windows they gave.

Second, the appliance data come from UK homes. The REFIT dataset was collected from 20 UK houses between 2013 and 2015, of which 18 are used, and may not reflect the

appliances and habits of Finnish households today; the windows taken from their usual run times carry UK habits into Finnish price days. No public Finnish dataset measures washing machines, dishwashers and tumble dryers separately over time. Fingrid's Datahub open data publish the hourly consumption of Finnish metering points only in aggregated form, by user group (Fingrid, 2024), and the Finnish household electricity use study reports measured annual consumption per appliance group from about one hundred homes, without load time series (Adato Energia, 2008). Appliance-level datasets from other countries, such as a Norwegian dataset of a single household over 24 days, are too small to replace REFIT. The cycle profiles should transfer, because the heating phase that dominates the energy and peak power of a wash cycle depends on the machine and not on the country. The base load transfers less well: many Finnish homes have electric space heating or a heat pump, whose power far exceeds that of the UK homes in REFIT, so in a real Finnish house the 8 kW limit would bind more often than in these scenarios.

Third, the car charge and the sauna are modelled rather than measured. Their power comes from product data, but the arrival and sauna times are assumptions, the same on every day, and both loads have constant power. The electrified load was also added after the exploratory results showed that the REFIT loads rarely reach 8 kW, and the length of the charge was set so that starting every appliance when ready stays valid; both choices were made before the held-out set was built and are recorded in the pre-registration.

Fourth, LLM output is non-deterministic, and two runs of the same scenario may produce different schedules. Each scenario is run three times and the results are averaged, which reduces but does not remove this variance; on the held-out days, however, every agent run returned the optimal schedule. Fifth, the daily appliance plan is given in structured form, with an explicit window for every appliance. The study therefore does not test whether the agents can interpret wishes stated informally, which is where they are expected to have an advantage over rule-based scheduling. On these fully specified problems, the smart timer, a simple rule written for this study, reached the optimum on every held-out day without a language model.

Sixth, the final design was evaluated with one commercial language model, GPT-6 Luna, through one cloud service, and the held-out run took place on a single day. Its response time therefore reflects that service on that day, and the results may differ with other models. The local open-weight model gpt-oss-20b was run only on the light and heavy days of the exploratory benchmark. Seventh, the peak limit was a hard constraint, whereas real power-based tariffs price the peak instead of forbidding it, and they measure it over 60 minutes rather than 15. Only the demonstration application lets a schedule exceed the limit, charging a fee for each kilowatt above it; none of the reported results use that fee. Eighth, the explanations that the pipeline gives were not evaluated with users, and the reasoning traces of the evaluation runs were not recorded, because tracing was switched off during them. The study shows that the pipeline produces a correct explanation of every schedule, not that households understand or trust it more than the plan of an optimizer.

6.5 Future research

Several directions for future research emerge from this study. First, extending the pipeline to thermal loads such as electric space heating, heat pumps and domestic hot water systems would address the gap explicitly identified by El Makroum, Zwickl-Bernhard, and Kranzl (2026). Second, the comparison of one agent with two should be repeated where the agents have more to decide: without the optimiser tool, so that the agents must stagger the appliances with the conflict tools, and with requests stated in ordinary language. These are the conditions under which a hierarchy of agents could still pay off. Third, a benchmark of informal household requests with annotated constraints would test the agents where a rule-based scheduler cannot operate, by measuring how accurately each system turns such requests into a correct schedule. Fourth, a live deployment study involving Finnish households on spot-price contracts would provide ecological validity that simulation-based evaluation cannot. Fifth, evaluating the design with other language models, including small open-weight models run locally on the full benchmark, would show whether its results depend on GPT-6 Luna; since the optimiser tool takes over the search, a small model may be sufficient for the remaining work of formulating, checking and explaining. Sixth, the hard peak limit of the benchmark could be replaced by the power charge of a real distribution tariff, so that a system weighs the energy saving of a cheap hour against the cost of raising the household's monthly peak, as under the Energy Authority's regulation (Energy Authority, 2026). The demonstration application of this thesis already prices the peak in this way, with an optimum that includes the fee, which could serve as a starting point. Seventh, implementing the price API and data reader as Model Context Protocol (MCP) servers (Anthropic, 2024) would enable reuse by future researchers without custom integration code. Eighth, a user study should test the explanations: whether households understand the pipeline's recommendations, trust them more than a numeric plan, and follow them, which is the advantage over an optimizer that this thesis argues for but did not measure.

7. Conclusions

Households on dynamic electricity contracts can lower their bills by running flexible appliances in the day's low-price hours, but planning several appliances against a changing price curve and a limit on the household's total power is hard to do every day without help. This thesis designed and evaluated, following Design Science Research, a hierarchical ReAct multi-agent system that gives a household a daily appliance recommendation. Two agents built in LangGraph, an Orchestrator and a Price-Aware Scheduler, work inside a deterministic harness that profiles the appliances, shows the Scheduler similar past days whose best schedules are known, and checks every schedule before it is returned; the Scheduler hands the search for the cheapest schedule to an optimiser tool. A single agent in the same harness was built for comparison. The system was evaluated on appliance cycles and usual run times measured in the REFIT homes and on Finnish day-ahead prices, under the 8 kW threshold of the Finnish power charge, first on an exploratory benchmark and then on a held-out benchmark of 72 scenarios with hypotheses registered in advance.

7.1 Answers to the research questions

The main **RQ** asked how such a system can be designed to give reliable, cost-saving daily recommendations. The system became reliable and optimal once the arithmetic was done by deterministic tools, the combinatorial search by a solver, and every schedule was checked independently of the agents, with a deterministic fallback for the rare case in which the model leaves an appliance out. Moving the fixed parts of the work into a deterministic harness reduced four agents to two without changing the schedules, and a single agent in the same harness was as reliable as the two, a third faster and cheaper. The language model's part is to formulate the problem, repair what the check reports and explain the result to the household; on these problems, one agent suffices for it.

SQ1 asked how much the recommendations save, how often they are valid, and what they cost to produce, compared with a language model answering without tools. On the held-out days the pipeline saved 15.9 % of the cost of starting every appliance when ready, the saving of the optimal schedule, in every run, against 4.1 % for the same model without tools. All 216 of its schedules were valid, against 77 % of those of the model without tools. A recommendation took 7.4 seconds on average and cost about $0.0004 in API fees. Tool use, not the language model alone, is what makes the recommendations valuable.

SQ2 asked under which price patterns and household loads the recommendations save at least as much as existing home energy management tools. The pipeline saved more than the cheapest-hours automations shared for Home Assistant under every price pattern and every load, by 4.6 and 6.0 percentage points per scenario on average; in homes with an electric car and a sauna, these automations, which plan each appliance on its own, broke the 8 kW limit on up to 46 % of the days. The advantage was not significantly larger

with the electrified load than with the lighter ones. The pipeline saved the same as the optimizer EMHASS under every condition. A simple peak-aware rule written for this study also reached the optimum, which shows that the advantage over the cheapest-hours tools comes from planning the appliances together under the limit.

7.2 Contributions

The thesis makes three contributions. The first is the artifact: the Multi-Agent Pipeline and its deterministic harness, together with the documented design iterations that led to it, the comparison with a single agent in the same harness, and six design principles drawn from them (Section 6.1). The second is an evaluation of what its recommendations are worth and cost, against a language model without tools, tested with pre-registered hypotheses on held-out days. The third is a map of the conditions under which an agent-based recommendation saves at least as much as the home energy management tools households can already install, built on a factorial benchmark that varies the price pattern and the household load separately under a sourced peak limit. The benchmarks, the case library, the code, the pre-registration and all run records are published in the project repository, so the results can be reproduced and extended.

7.3 Implications

For research on language model agents, the results add to the evidence that agents are most reliable when they use solvers and checks rather than replace them, and they show that once they do, a hierarchy of agents may add cost without adding reliability. They also show that comparing an agent only with a language model without tools, or only with an optimum, hides the most relevant comparison: the simple tools and rules that users already have. For practice in Finland, where a third of households had a dynamic-price contract at the end of 2024 (Energy Authority, 2025) and network companies are moving towards power-based distribution tariffs, the results suggest that the cheapest-hours automations popular today will become costly in homes with an electric car or a sauna, and that any recommendation should account for the household's total power. An agent-based system can match an optimizer on cost at a negligible API cost per day, and its value over an optimizer lies in the explanation and in accepting the household's plan without numeric configuration.

7.4 Limitations and further work

The evaluation is a simulation based on UK appliance data, modelled car and sauna loads and one commercial language model, with the peak limit as a hard constraint, and the explanations were not tested with users (Section 6.4). The most important next steps are a comparison of one agent with several where the agents have more to decide, a field

study with Finnish households, a test of whether households understand and act on the explanations, a benchmark of wishes stated in ordinary language, where a rule-based tool cannot follow, and a version of the problem in which the peak is priced as in a real power-based tariff (Section 6.5).

References

Aalto University. (2026, March 23). *Energy poverty is not recognized in Finland* [News release on a study by S. Numminen and S. Karhinen, Aalto University and Finnish Environment Institute]. https://www.aalto.fi/en/news/energy-poverty-is-not-recognized-in-finland

Aamodt, A., & Plaza, E. (1994). Case-based reasoning: Foundational issues, methodological variations, and system approaches. *AI Communications, 7*(1), 39–59. https://doi.org/10.3233/AIC-1994-7104

Adato Energia. (2008). *Kotitalouksien sähkönkäyttö 2006* [Household electricity use 2006]. Adato Energia Oy. ISBN 978-952-9696-41-3.

Amershi, S., Weld, D., Vorvoreanu, M., Fourney, A., Nushi, B., Collisson, P., Suh, J., Iqbal, S., Bennett, P. N., Inkpen, K., Teevan, J., Kikin-Gil, R., & Horvitz, E. (2019). Guidelines for human-AI interaction. In *Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems*. ACM. https://doi.org/10.1145/3290605.3300233

Anthropic. (2024). *Model Context Protocol*. https://modelcontextprotocol.io

Beaudin, M., & Zareipour, H. (2015). Home energy management systems: A review of modelling and complexity. *Renewable and Sustainable Energy Reviews, 45*, 318–335. https://doi.org/10.1016/j.rser.2015.01.046

Belcak, P., Heinrich, G., Diao, S., Fu, Y., Dong, X., Muralidharan, S., Lin, Y. C., & Molchanov, P. (2025). *Small language models are the future of agentic AI* [Position paper, NVIDIA Research]. arXiv:2506.02153. https://doi.org/10.48550/arXiv.2506.02153

Cali, U., Halden, U., Andoni, M., Catak, F. O., Chen, S., Couraud, B., Kantar, E., Knapper, S., Kucukdemiral, I., Kusetogullari, H., Kuzlu, M., Mousavi, Y., Norbu, S., Ustun, T. S., & Flynn, D. (2026). Generative AI and LLM applications in renewable energy and smart grids: A systematic review for the sustainable energy transition. *Artificial Intelligence Review, 59*(7), 153. https://doi.org/10.1007/s10462-026-11545-2

Caruna. (2026, March 20). *The strategic stockpile fee included in the electricity tax will increase from 1 April 2026* [News release]. https://caruna.fi/en/news/strategic-stockpile-fee-included-electricity-tax-will-increase-1-april-2026

Daniel. (2022). *Nordpool Planner* [Home Assistant custom component]. GitHub. Retrieved 5 October 2026, from https://github.com/dala318/nordpool_plan

ner

Dolinin, E., & Kiourt, C. (2026). Multi-agent-based smart-home energy management with adaptive reasoning. *Applied Sciences, 16*(4), 1896. https://doi.org/10.3390/app16041896

El Makroum, R., Zwickl-Bernhard, S., & Kranzl, L. (2026). Agentic AI home energy management system: A large language model framework for residential load scheduling. *Results in Engineering, 29*, 109857. https://doi.org/10.1016/j.rineng.2026.109857

El Makroum, R., Zwickl-Bernhard, S., Kranzl, L., & Auer, H. (2026). *Conversational demand response: Bidirectional aggregator-prosumer coordination through agentic AI* [Preprint]. arXiv:2603.06217. https://arxiv.org/abs/2603.06217

Elenia. (2026). *Connection price list for places of electricity use, 1 January 2026*. Retrieved 15 October 2026, from https://www.elenia.fi/files/8ced42591a7e0ca3b6f608cea8d548451c9fb041/elenia-liittymishinnasto-2026-en.pdf

Elering AS. (2026). *Nord Pool day-ahead prices, bidding zone FI* [Data service; data owner Nord Pool]. Retrieved 26 September 2026, from https://dashboard.elering.ee/api/nps/price

ELY Centre (ELY-keskus). (n.d.). *Ekotekoja: Sauna ja mökkeily* [Eco-actions: Sauna and cottage life]. Retrieved 20 October 2026, from https://ely-keskus.fi/documents/10191/56418/Ekotekoja+sauna+ja+mökkeily/fffe9c9a-be03-410c-84f8-726badbb2cea

EMHASS contributors. (2026). *EMHASS: Energy Management for Home Assistant* (Version 0.18.4) [Computer software]. https://emhass.readthedocs.io

Energy Authority (Energiavirasto). (2025, July 10). *Dynamic-price contracts held by more than one third of households: Fixed-term contracts still the most common type in 2024* [Press release]. https://energiavirasto.fi/en/-/dynamic-price-contracts-held-by-more-than-one-third-of-households-fixed-term-contracts-still-the-most-common-type-in-2024

Energy Authority (Energiavirasto). (2026). *Määräys sähkönjakelupalvelutuotteiden maksukomponenttien määräytymisperusteista* [Regulation on the determination criteria for the payment components of electricity distribution service products] (Dnro 2340/000002/2025). Finlex. https://www.finlex.fi/api/media/authority-regulation/911686/mainPdf/main.pdf

ENTSO-E. (2026). *Transparency Platform: Day-ahead prices, bidding zones FI and GB* [Data set]. European Network of Transmission System Operators for Electricity. Retrieved 6 October 2026, from https://transparency.entsoe.eu

Fingrid. (2024). *Fingrid's Open data service now provides data on electricity consumption and small-scale electricity production from accounting points in*

Finland [News release]. Retrieved 5 October 2026, from https://www.fingrid.fi/en/news/news/2024/fingrids-open-data-service-now-provides-data-on-electricity-consumption-and-small-scale-electricity-production-from-accounting-points-in-finland/

Finnish Tax Administration. (2026). *Rates of VAT*. Retrieved 15 October 2026, from https://vero.fi/en/businesses-and-corporations/taxes-and-charges/vat/rates-of-vat/

Ghafari, M., Sami, A., Rezapour, H., Saravani Ghayour, S., Lin, F., Basaran, K., Lazaroiu, G. C., & Siano, P. (2025). A comprehensive review on the application of large language models (LLMs) in power systems. *IEEE Access, 13*, 209450–209486. https://doi.org/10.1109/ACCESS.2025.3637226

Giudici, M., Padalino, L., Paolino, G., Paratici, I., Pascu, A. I., & Garzotto, F. (2024). Designing home automation routines using an LLM-based chatbot. *Designs, 8*(3), 43. https://doi.org/10.3390/designs8030043

Gkalinikis, N. V., Nalmpantis, C., Vrakas, D., Chatzigeorgiou, S., Athanasiadis, C., & Doukas, D. (2025). RHEA: Residential home energy advisor. In *2025 10th International Conference on Smart and Sustainable Technologies (SpliTech)* (pp. 1–6). IEEE. https://doi.org/10.23919/SpliTech65624.2025.11091692

Gokhale, G., Claessens, B., & Develder, C. (2024). Explainable home energy management systems based on reinforcement learning using differentiable decision trees. In *Proceedings of the 15th ACM International Conference on Future and Sustainable Energy Systems (e-Energy '24)* (pp. 687–691). ACM. https://doi.org/10.1145/3632775.3663310

Gonçalves, V. P., Serrano, A. L. M., Rodrigues, G. A. P., de Oliveira, M. N., Meneguette, R. I., Bispo, G. D., Peixoto, M. G. M., & Filho, G. P. R. (2025). An empirically validated framework for automated and personalized residential energy-management integrating large language models and the Internet of Energy. *Energies, 18*(14), 3744. https://doi.org/10.3390/en18143744

Gregor, S., & Hevner, A. R. (2013). Positioning and presenting design science research for maximum impact. *MIS Quarterly, 37*(2), 337–355. https://doi.org/10.25300/MISQ/2013/37.2.01

Guo, T., Chen, X., Wang, Y., Chang, R., Pei, S., Chawla, N. V., Wiest, O., & Zhang, X. (2024). Large language model based multi-agents: A survey of progress and challenges. In *Proceedings of the 33rd International Joint Conference on Artificial Intelligence (IJCAI 2024)* (pp. 8048–8057).

Haakana, J., Haapaniemi, J., Lamminpää, A., Lassila, J., Viiliäinen, S., & Vähäsaari, M. (2025). Effects of extreme spot market prices on the electrical power system: Lessons from the Finnish distribution system. *IET Generation, Transmission & Distribution*, e70114. https://doi.org/10.1049/gtd2.70114

Han, S., Zhang, Q., Jin, W., & Xu, Z. (2024). *LLM multi-agent systems: Challenges and open problems* [Preprint]. arXiv:2402.03578. https://doi.org/10.48550/arXiv.2402.03578

Helen Sähköverkko. (2026a). *Network service products and prices*. Retrieved 4 October 2026, from https://www.helensahkoverkko.fi/en/private-customers/Information-about-electricity-consumption/Network-service-products-and-prices

Helen Sähköverkko. (2026b, February 16). *Energy Authority clarifies electricity distribution pricing: an option without a power charge remains for households* [News release]. https://www.helensahkoverkko.fi/en/uutiset/2026/energy-authority-clarifies-electricity-distribution-pricing

Hevner, A. R., March, S. T., Park, J., & Ram, S. (2004). Design science in information systems research. *MIS Quarterly, 28*(1), 75–105.

Hong, S., Zhuge, M., Chen, J., Zheng, X., Cheng, Y., Wang, J., Zhang, C., Wang, Z., Yau, S. K. S., Lin, Z., Zhou, L., Ran, C., Xiao, L., Wu, C., & Schmidhuber, J. (2024). MetaGPT: Meta programming for a multi-agent collaborative framework. In *International Conference on Learning Representations (ICLR 2024)*. https://doi.org/10.48550/arXiv.2308.00352

Jahanbakhsh, N., Vega-Barbas, M., Pau, I., Elvira-Martín, L., Moosavi, H., & García-Vázquez, C. (2025). Leveraging retrieval-augmented generation for automated smart home orchestration. *Future Internet, 17*(5), 198. https://doi.org/10.3390/fi17050198

Jonah, S., Moses, Q., Babatunde, A., Ajao-Olarinoye, M., & Bammeke, D. (2026). LLMs for agentic home energy management [Preprint]. arXiv:2607.04569. https://doi.org/10.48550/arXiv.2607.04569

Jung, W. (2026). Multi-agent home energy management assistant (HEMA). *SoftwareX, 34*, 102633. https://doi.org/10.1016/j.softx.2026.102633

Kert, K. (2023). *Nordpool cheapest hours actions* [Home Assistant blueprint]. GitHub. Retrieved 3 October 2026, from https://github.com/kullarkert/ha_nordpool_hours_blueprint

King, E., Yu, H., Lee, S., & Julien, C. (2024). Sasha: Creative goal-oriented reasoning in smart homes with large language models. *Proceedings of the ACM on Interactive, Mobile, Wearable and Ubiquitous Technologies, 8*(1), Article 12. https://doi.org/10.1145/3643505

Kovacs, A., Meteier, Q., Angelini, L., Mugellini, E., & Abou Khaled, O. (2025). Explanation modalities for a recommender system optimizing energy management in a solar-powered smart home. In *Companion Proceedings of the 30th International Conference on Intelligent User Interfaces* (pp. 43–47). ACM.

https://doi.org/10.1145/3708557.3716331

K-Rauta. (2026). *Sähkökiuas Harvia Virta HL70* [Electric sauna heater Harvia Virta HL70, product page]. Retrieved 20 October 2026, from https://www.k-rauta.fi/tuote/sahkokiuas-harvia-virta-hl70/6410082613113

Lakens, D. (2017). Equivalence tests: A practical primer for *t* tests, correlations, and meta-analyses. *Social Psychological and Personality Science, 8*(4), 355–362. https://doi.org/10.1177/1948550617697177

Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W., Rocktäschel, T., Riedel, S., & Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems 33 (NeurIPS 2020)* (pp. 9459–9474).

Li, B., Mellou, K., Zhang, B., Pathuri, J., & Menache, I. (2023). *Large language models for supply chain optimization* [Preprint]. arXiv:2307.03875. https://doi.org/10.48550/arXiv.2307.03875

Liu, S., Hu, R., Yang, Y., Zhang, H., Ding, X., Sun, H., & Sun, L. (2025). LLM-BEM: Zero-shot LLM-agent scheduling for smart building energy management. In *2025 IEEE 26th China Conference on System Simulation Technology and its Applications (CCSSTA)* (pp. 69–74). IEEE. https://doi.org/10.1109/IEEECONF65522.2025.11137227

Luccioni, S., Jernite, Y., & Strubell, E. (2024). Power hungry processing: Watts driving the cost of AI deployment? In *Proceedings of the 2024 ACM Conference on Fairness, Accountability, and Transparency (FAccT '24)* (pp. 85–99). https://doi.org/10.1145/3630106.3658542

March, S. T., & Smith, G. F. (1995). Design and natural science research on information technology. *Decision Support Systems, 15*(4), 251–266.

Michelon, F., Zhou, Y., & Morstyn, T. (2025). Large language model interface for home energy management systems. In *Proceedings of the 16th ACM International Conference on Future and Sustainable Energy Systems (e-Energy '25)* (pp. 590–602). ACM. https://doi.org/10.1145/3679240.3734586

Mocanu, E., Mocanu, D. C., Nguyen, P. H., Liotta, A., Webber, M. E., Gibescu, M., & Slootweg, J. G. (2019). On-line building energy optimization using deep reinforcement learning. *IEEE Transactions on Smart Grid, 10*(4), 3698–3708. https://doi.org/10.1109/TSG.2018.2834219

Moghanjooghi, A. A., Mazloomi, M., Asgari, M., & Mehrabikia, M. S. (2025). HyLL: Hybrid LSTM and large language model-based recommendation system for energy management and consumption optimization in smart homes. In *2025 6th International Conference on Optimizing Electrical Energy Consumption (OEEC)* (pp. 1–6). IEEE. https://doi.org/10.1109/OEEC66525.2025.11100170

Mongaillard, T., Lasaulce, S., Hicheur, O., Zhang, C., Bariah, L., Varma, V. S., Zou, H., Zhao, Q., & Debbah, M. (2024). *Large language models for power scheduling: A user-centric approach* [Preprint]. arXiv:2407.00476. https://doi.org/10.48550/arXiv.2407.00476

Moradzadeh, B., & Tomsovic, K. (2013). Two-stage residential energy management considering network operational constraints. *IEEE Transactions on Smart Grid, 4*(4), 2339–2346. https://doi.org/10.1109/TSG.2013.2265313

Motiva. (2026). *Kiinteistön energiatehokkaat sähkötekniset ratkaisut* [Energy-efficient electrical solutions for buildings]. Retrieved 20 October 2026, from https://www.motiva.fi/wp-content/uploads/sites/2/2026/01/Kiinteiston-energiatehokkaat-sahkotekniset-ratkaisut.pdf

Murray, D., & Stankovic, L. (2016). *REFIT: Electrical load measurements (cleaned)* [Data set]. University of Strathclyde. https://doi.org/10.15129/9ab14b0e-19ac-4279-938f-27f643078cec

Murray, D., Stankovic, L., & Stankovic, V. (2017). An electrical load measurements dataset of United Kingdom households from a two-year longitudinal study. *Scientific Data, 4*, 160122. https://doi.org/10.1038/sdata.2016.122

OpenHEMS contributors. (2026). *OpenHEMS: Home energy management system based on Home Assistant* [Computer software]. GitHub. Retrieved 5 October 2026, from https://github.com/abriotde/openhems-sample

OpenAI. (2026). *Pricing*. OpenAI API documentation. Retrieved 21 October 2026, from https://developers.openai.com/api/docs/pricing

Oprea, S.-V., & Bâra, A. (2024). A recommendation system for prosumers based on large language models. *Sensors, 24*(11), 3530. https://doi.org/10.3390/s24113530

Oulun Energia Sähköverkko. (2026). *Network service price list from January 1, 2026*. Retrieved 15 October 2026, from https://www.oulunenergia.fi/globalassets/tiedostot/sahkoverkot/oes_network_service_pricelist_for_january_1_2026.pdf

Page, M. J., McKenzie, J. E., Bossuyt, P. M., Boutron, I., Hoffmann, T. C., Mulrow, C. D., Shamseer, L., Tetzlaff, J. M., Akl, E. A., Brennan, S. E., Chou, R., Glanville, J., Grimshaw, J. M., Hróbjartsson, A., Lalu, M. M., Li, T., Loder, E. W., Mayo-Wilson, E., McDonald, S., … Moher, D. (2021). The PRISMA 2020 statement: An updated guideline for reporting systematic reviews. *BMJ, 372*, n71. https://doi.org/10.1136/bmj.n71

Pakastin Oy. (2026). *Sähkötin: Nord Pool FI day-ahead spot prices* [Data service]. Retrieved 25 September 2026, from https://sahkotin.fi

Papaioannou, A., Dimara, A., & Krinidis, S. (2025). GUIDE: A prescriptive hybrid AI framework for energy-efficient appliances usage through behavioral modeling and

LLM guidance. *Energy and Buildings, 348*, 116463. https://doi.org/10.1016/j.enbuild.2025.116463

Papaioannou, I., Korkas, C., & Kosmatopoulos, E. (2025). Smart building recommendations with LLMs: A semantic comparison approach. *Buildings, 15*(13), 2303. https://doi.org/10.3390/buildings15132303

Peffers, K., Tuunanen, T., Rothenberger, M. A., & Chatterjee, S. (2007). A design science research methodology for information systems research. *Journal of Management Information Systems, 24*(3), 45–77.

Raghavan, V. S., & Giridhar, A. V. (2025). Cost-optimal residential energy scheduling via a zero-shot LLM agent and model predictive control. *Journal of Building Performance Simulation, 19*(3), 492–510. https://doi.org/10.1080/19401493.2025.2605465

Raghavan, V. S., & Giridhar, A. V. (2026). Integrating large language models into peer-to-peer energy management for multi-tenant buildings: A guardrail approach to ensuring resilience. *Neural Networks, 200*, 108814. https://doi.org/10.1016/j.neunet.2026.108814

Rashed, G. I., Bahageel, A. O. M., Gony, H. A. I., Badjan, A., & Shaheen, H. I. (2025). AI-driven multi-agent demand response framework for residential load optimization using CrewAI. In *2025 IEEE 20th Conference on Industrial Electronics and Applications (ICIEA)* (pp. 1–10). IEEE. https://doi.org/10.1109/ICIEA65512.2025.11149013

Schuirmann, D. J. (1987). A comparison of the two one-sided tests procedure and the power approach for assessing the equivalence of average bioavailability. *Journal of Pharmacokinetics and Biopharmaceutics, 15*(6), 657–680. https://doi.org/10.1007/BF01068419 Siino, M., & Tinnirello, I. (2024). GPT prompt engineering for scheduling appliances usage for energy cost optimization. In *2024 IEEE International Symposium on Measurements & Networking (M&N)* (pp. 1–6). IEEE. https://doi.org/10.1109/MN60932.2024.10615758

Sou, K. C., Weimer, J., Sandberg, H., & Johansson, K. H. (2011). Scheduling smart home appliances using mixed integer linear programming. In *Proceedings of the 50th IEEE Conference on Decision and Control and European Control Conference* (pp. 5144–5149). IEEE. https://doi.org/10.1109/CDC.2011.6161081

Venable, J., Pries-Heje, J., & Baskerville, R. (2016). FEDS: A framework for evaluation in design science research. *European Journal of Information Systems, 25*(1), 77–89. https://doi.org/10.1057/ejis.2014.36

Traficom. (n.d.). *Henkilöliikennetutkimus 2021* [National travel survey 2021, brochure]. Retrieved 20 October 2026, from https://traficom.fi/files/media/file/HLT2021_esite_FIN.pdf

Yang, Y., Yang, Q., Li, D., Sun, L., Liu, S., & Liu, R. (2025). LLM-MAC: LLM-driven multi-agent coordination for generalizable home energy management. In *2025 44th Chinese Control Conference (CCC)* (pp. 7318–7323). IEEE. https://doi.org/10.23919/CCC64809.2025.11179521

Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2023). ReAct: Synergizing reasoning and acting in language models. *International Conference on Learning Representations (ICLR 2023)*. arXiv:2210.03629.

Zhang, L., & Chen, Z. (2024). Large language model-based interpretable machine learning control in building energy systems. *Energy and Buildings, 313*, 114278. https://doi.org/10.1016/j.enbuild.2024.114278

Zharova, A., Boer, A., Knoblauch, J., Schewina, K. I., & Vihs, J. (2024). An explainable multi-agent recommendation system for energy-efficient decision support in smart homes. *Environmental Data Science, 3*, e7. https://doi.org/10.1017/eds.2024.8

Appendix A. Literature search

The literature review in Chapter 2 draws on a structured search of Scopus and IEEE Xplore, reported following the PRISMA 2020 statement (Page et al., 2021). Scopus was searched on 12 October 2026 in titles, abstracts and keywords, limited to articles and conference papers in English:

```
TITLE-ABS-KEY ( ( "large language model*" OR llm* OR "agentic
AI" OR "language agent*" ) AND ( "home energy management" OR
"demand response" OR "smart home*" OR ( ( appliance* OR load* )
W/3 schedul* ) ) ) AND PUBYEAR > 2019
```

IEEE Xplore was searched on 13 October 2026 in all metadata:

```
("All Metadata":"large language model*" OR "All Metadata":LLM*
OR "All Metadata":"agentic AI" OR "All Metadata":"language
agent*") AND ("All Metadata":"home energy management" OR "All
Metadata":"demand response" OR "All Metadata":"appliance
scheduling" OR "All Metadata":"smart home*")
```

The searches returned 608 records, 358 from Scopus and 250 from IEEE Xplore (Figure A.1); the Scopus search returned 357 records on its first run and 358 when it was rerun for screening. Their titles were screened against one criterion: a language model or language model agent applied to household or building energy management, demand response or appliance scheduling. This excluded 540 records, and removing 14 duplicates left 54 reports, to which two studies known to the author were added. Of the 56 reports assessed, 7 were excluded (four use no language model, two do not concern energy, one is an earlier version of an included study). Of the 49 eligible studies, 25 are discussed in Chapter 2; Appendix B lists all 49.

The procedure deviates from PRISMA 2020 in three respects. Duplicates were identified only among the records that passed title screening, because complete record exports were not available, so the count of excluded records may include duplicates; 47 of the 56 eligibility decisions rest on abstracts rather than full texts; and one reviewer screened the records with the help of an AI assistant, so reliability was not checked. The references on design science, agent architectures, statistics and the Finnish electricity market were collected separately.

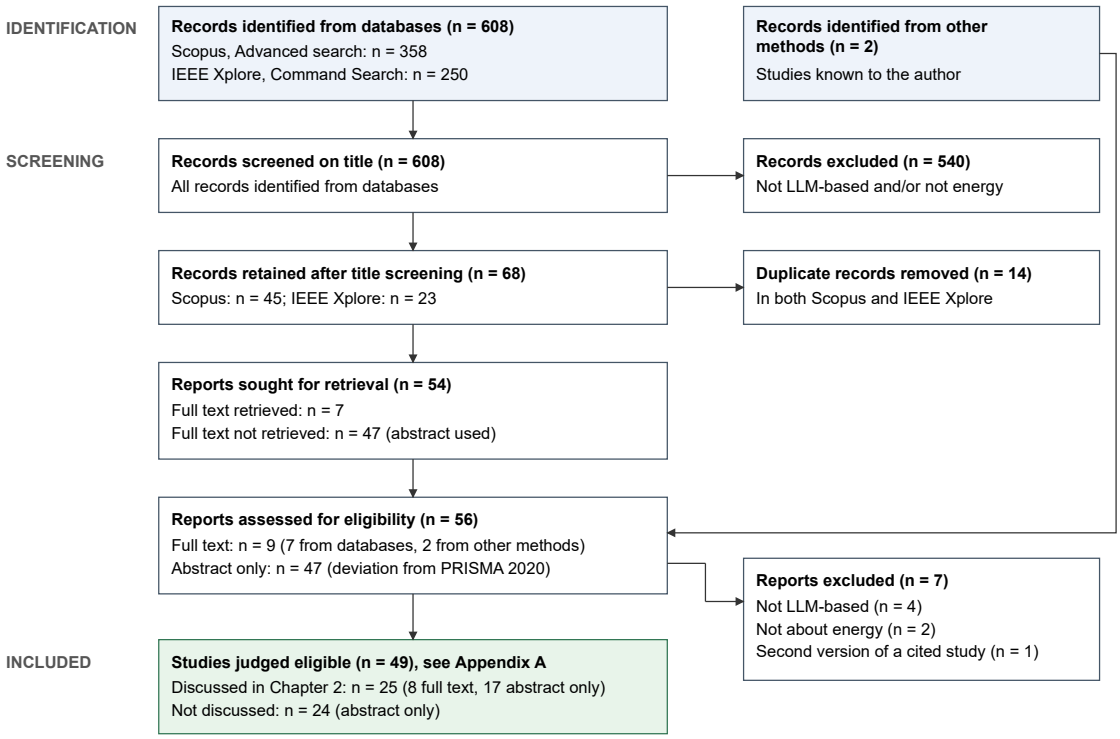


Figure A.1. PRISMA 2020 flow diagram of the literature search.

Appendix B. Eligible studies

The literature search (Appendix A) found 49 eligible studies. Table B.1 lists the 25 discussed in Chapter 2; they are in the reference list. Table B.2 lists the 24 that are not discussed. They are not cited in the text and are therefore identified by title and DOI instead of a reference list entry. All 24 were assessed on their abstracts only.

Table B.1 Eligible studies discussed in Chapter 2.

Study	Title	Assessed on
Cali et al. (2026)	Generative AI and LLM applications in renewable energy and smart grids: a systematic review for the sustainable energy transition	Abstract
Dolinin and Kiourt (2026)	Multi-Agent-Based Smart-Home Energy Management with Adaptive Reasoning	Abstract
El Makroum, Zwickl-Bernhard, Kranzl, and Auer (2026)	Conversational Demand Response: Bidirectional Aggregator-Prosumer Coordination through Agentic AI	Abstract
El Makroum, Zwickl-Bernhard, and Kranzl (2026)	Agentic AI home energy management system: A large language model framework for residential load scheduling	Abstract
Ghafari et al. (2025)	A Comprehensive Review on the Application of Large Language Models (LLMs) in Power Systems	Abstract
Giudici et al. (2024)	Designing Home Automation Routines Using an LLM-Based Chatbot	Full text
Gkalinikis et al. (2025)	RHEA: Residential Home Energy Advisor	Abstract
Gonçalves et al. (2025)	An Empirically Validated Framework for Automated and Personalized Residential Energy-Management Integrating Large Language Models and the Internet of Energy	Abstract
Jahanbakhsh et al. (2025)	Leveraging Retrieval-Augmented Generation for Automated Smart Home Orchestration	Full text
Jung (2026)	Multi-agent home energy management assistant (HEMA)	Abstract
King et al. (2024)	Sasha: Creative Goal-Oriented Reasoning in Smart Homes with Large Language Models	Full text
Kovacs et al. (2025)	Explanation Modalities for a Recommender System Optimizing Energy Management in a Solar-Powered Smart Home	Abstract
Liu et al. (2025)	LLM-BEM: Zero-shot LLM-Agent Scheduling for Smart Building Energy Management	Abstract
Michelon et al. (2025)	Large Language Model Interface for Home Energy Management Systems	Full text

Study	Title	Assessed on
Moghanjooghi et al. (2025)	HyLL: Hybrid LSTM and Large Language Model-Based Recommendation System for Energy Management and Consumption Optimization in Smart Homes	Abstract
Mongaillard et al. (2024)	Large Language Models for Power Scheduling: A User-Centric Approach	Abstract
Oprea and Bâra (2024)	A Recommendation System for Prosumers Based on Large Language Models	Full text
A. Papaioannou et al. (2025)	GUIDE: A prescriptive hybrid AI framework for energy-efficient appliances usage through behavioral modeling and LLM guidance	Full text
I. Papaioannou et al. (2025)	Smart Building Recommendations with LLMs: A Semantic Comparison Approach	Full text
Raghavan and Giridhar (2025)	Cost-optimal residential energy scheduling via a zero-shot LLM agent and model predictive control	Abstract
Raghavan and Giridhar (2026)	Integrating large language models into Peer-to-Peer energy management for multi-tenant buildings: A guardrail approach to ensuring resilience	Abstract
Rashed et al. (2025)	AI-Driven Multi-Agent Demand Response Framework for Residential Load Optimization Using CrewAi	Abstract
Siino and Tinnirello (2024)	GPT Prompt Engineering for Scheduling Appliances Usage for Energy Cost Optimization	Abstract
Yang et al. (2025)	LLM-MAC: LLM-Driven Multi-Agent Coordination for Generalizable Home Energy Management	Abstract
Zhang and Chen (2024)	Large language model-based interpretable machine learning control in building energy systems	Full text

Table B.2 Eligible studies not discussed in the text.

Year	Title	DOI
2026	A Large Language Model-Based Agent Framework for Simulating Building Users' Air-Conditioning Setpoint Adjustment Behavior Under Demand Response	10.3390/buildings16050887
2026	A Method for Characterizing the Adjustable Potential Features of Air Conditioning Loads Based on LLM Agents	10.1109/iceemt71099.2026.11686325
2026	A smart home platform integrating an energy-aware digital twin and an LLM-enabled conversational agent	10.1080/0144929x.2026.2672579

Year	Title	DOI
2026	Adaptive home energy management to self-motivated user preferences via iterative LLM-augmented reinforcement learning	10.1016/j.apenergy.2026.127976
2025	AI-driven smart home energy optimization: integrating AI agents with IoT for adaptive decision-making	10.1049/icp.2025.2536
2025	An LLM Framework for Inferring Household Energy Consumption Through Behaviour Simulation	10.1145/3736425.3770093
2023	Assessing LLMs Responses in the Field of Domestic Sustainability: An Exploratory Study	10.1109/ddp60485.2023.00019
2025	BitRL-Light: 1-Bit LLM Agents with Deep Reinforcement Learning for Energy-Efficient Smart Home Lighting Optimization	10.1109/ipccc66453.2025.11304691
2025	Data and Knowledge–Driven Approach for Energy Profiling in Smart Context-Aware Buildings	10.1007/978-3-032-11402-0_6
2025	FlexiDigital: A Comprehensive Approach to Energy Flexibility Services using Digital Twins and Large Language Models	10.1145/3679240.3734662
2025	Implementing AI in Smart Buildings: A Modular, Proof-of-Concept Approach	10.1109/epec65543.2025.11230379
2025	Integrating AI and IoT for Smart Home Energy Management	10.1109/icitr69413.2025.11353739
2026	Integrating Large Language Models and Reinforcement Learning for Efficient Home Energy Management	10.1109/jiot.2026.3719299
2026	Large Language Model Assisted Peer-to-Peer Energy Trading System	10.1061/jleed9.eyeng-6440
2026	Large Language Model Driven Two-Layer Demand Response Framework for Electric Vehicle and Aggregators	10.1007/978-981-92-1257-6_8
2026	Large language model-enhanced home energy management with dynamic user preference elicitation and hierarchical data-sharing	10.1016/j.apenergy.2026.127540
2026	Large Language Models in Home Energy and Comfort Management: Opportunities, Framework, and Challenges	10.46855/energy-proceedings-12239
2026	LLM-Assisted Edge Intelligence Framework for Smart Home Energy Management	10.1109/apan70967.2026.11709693
2026	Personalized Home Energy Management Using Non-Intrusive Load Monitoring and Human-in-the-Loop Reinforcement Learning	10.1109/pesgm58988.2026.11693075
2024	Privacy-Preserving Energy Recommendations Using Federated Learning and Local LLMs on the Edge	10.1109/bdcat63179.2024.00065

Year	Title	DOI
2025	Review of optimization control methods for HVAC systems in Demand Response (DR): Transition from model-driven to model-free approaches and challenges	10.1016/j.buildenv.2025.113045
2025	Sustainable Energy Management via Integrating Model Context Protocol with Home Assistant	10.1109/isgtmiddleeast65737.2025.11314322
2026	Toward a Unified Architecture for Smart Home Energy Monitoring: Requirements, Design, and Use-Case Validation	10.1109/ccnc65079.2026.11366485
2026	What LLM-Simulated Users Can and Cannot Tell Us About Conversational Energy Management Systems	10.1145/3765611.3815484